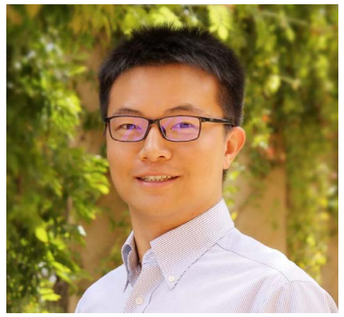
MIT HAN LAB

EfficientML.ai Lecture 06 Quantization

Part II



Song Han

Associate Professor, MIT
Distinguished Scientist, NVIDIA

@SongHan_MIT



Lecture Plan

Today we will:

1. Review Linear Quantization.

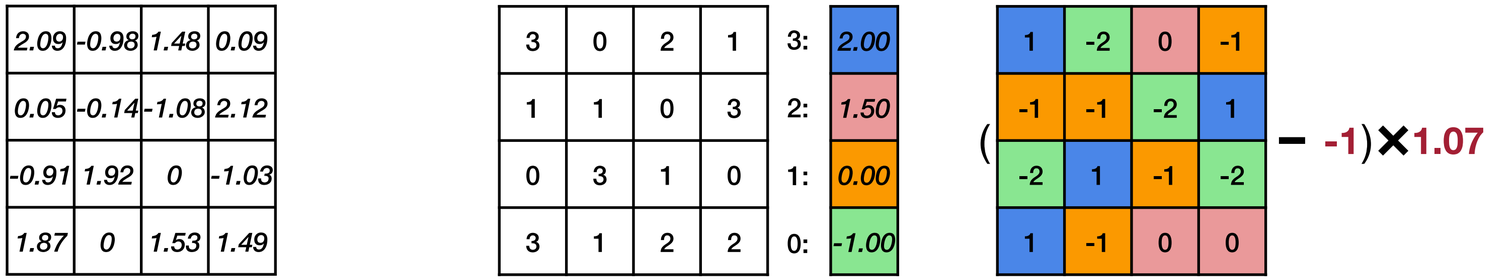
2. Introduce **Post-Training Quantization (PTQ)** that quantizes a floating-point neural network model, including: channel quantization, group quantization, and range clipping.

3. Introduce **Quantization-Aware Training (QAT)** that emulates inference-time quantization during the training/fine-tuning and recover the accuracy.

4. Introduce **binary and ternary** quantization.

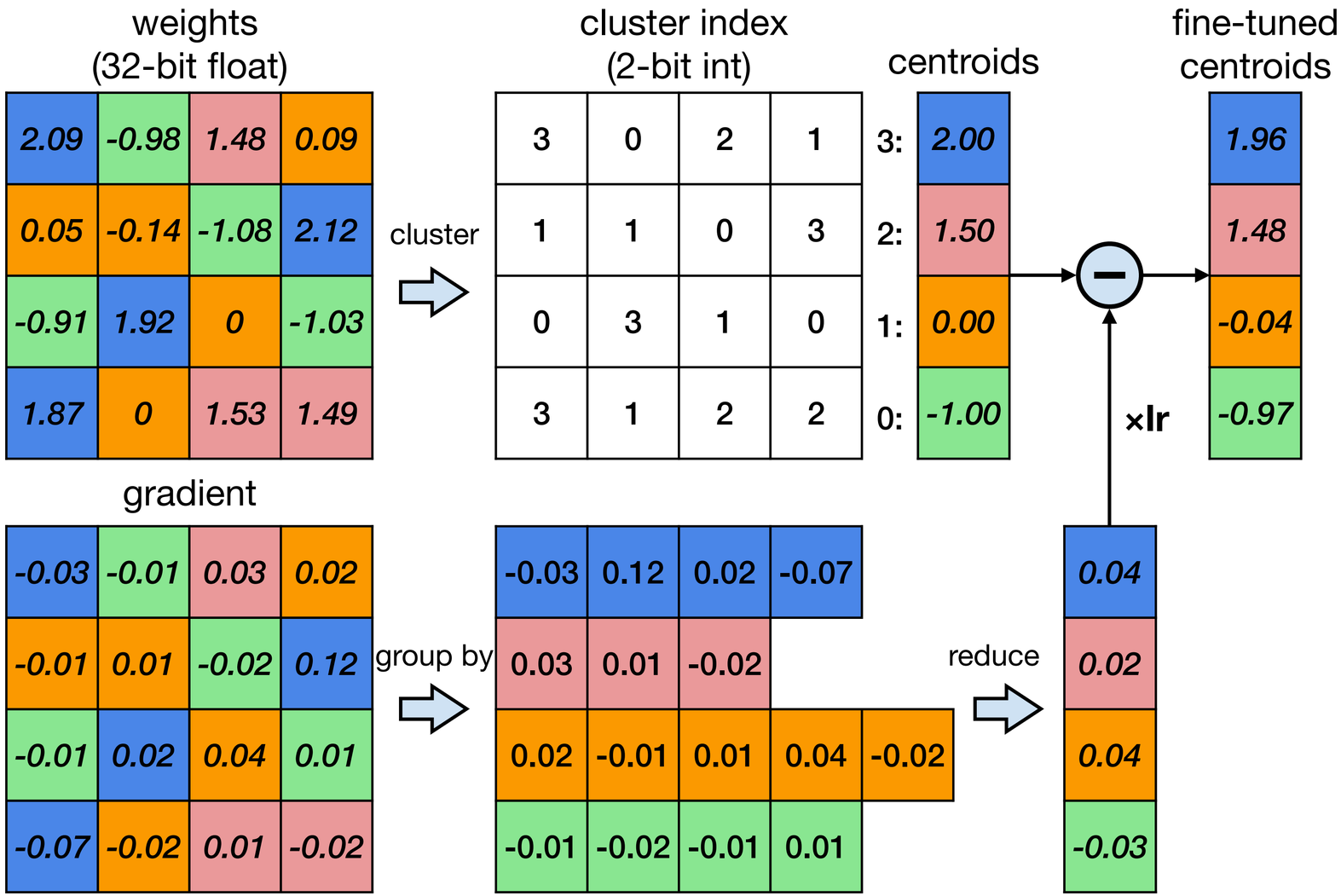
5. Introduce automatic **mixed-precision** quantization.

Neural Network Quantization



		K-Means-based Quantization	**Linear Quantization**
Storage	Floating-Point Weights	Integer Weights; Floating-Point Codebook	Integer Weights
Computation	Floating-Point Arithmetic	Floating-Point Arithmetic	Integer Arithmetic

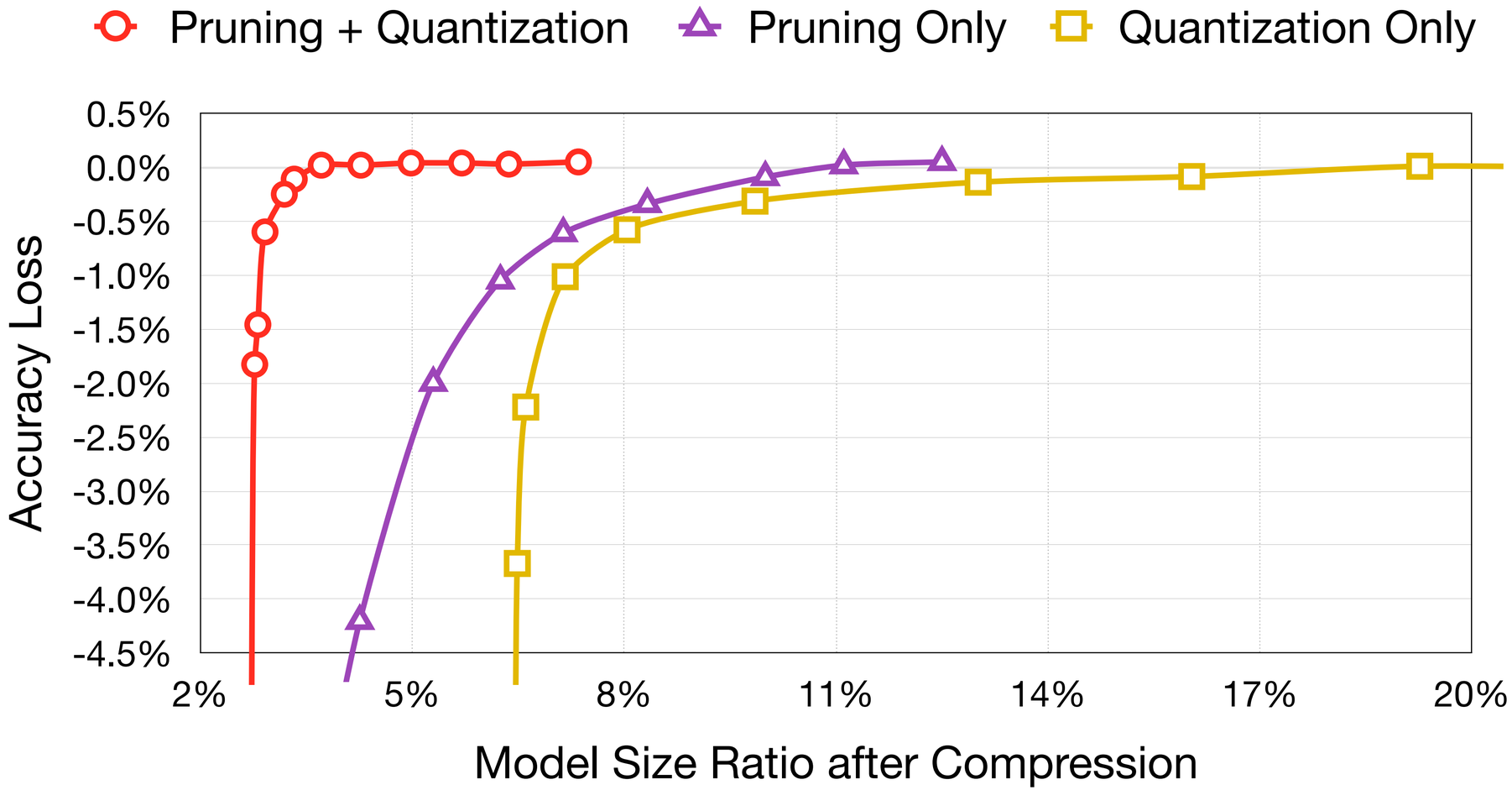
K-Means-based Weight Quantization



Deep Compression [Han *et al.*, ICLR 2016]

K-Means-based Weight Quantization

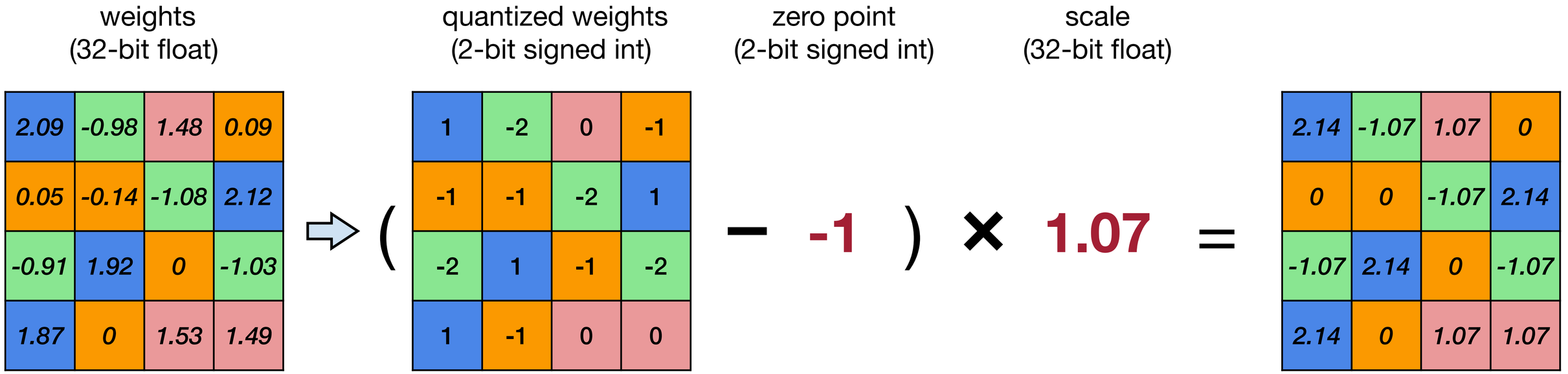
Accuracy vs. compression rate for AlexNet on ImageNet dataset



Deep Compression [Han *et al.*, ICLR 2016]

Linear Quantization

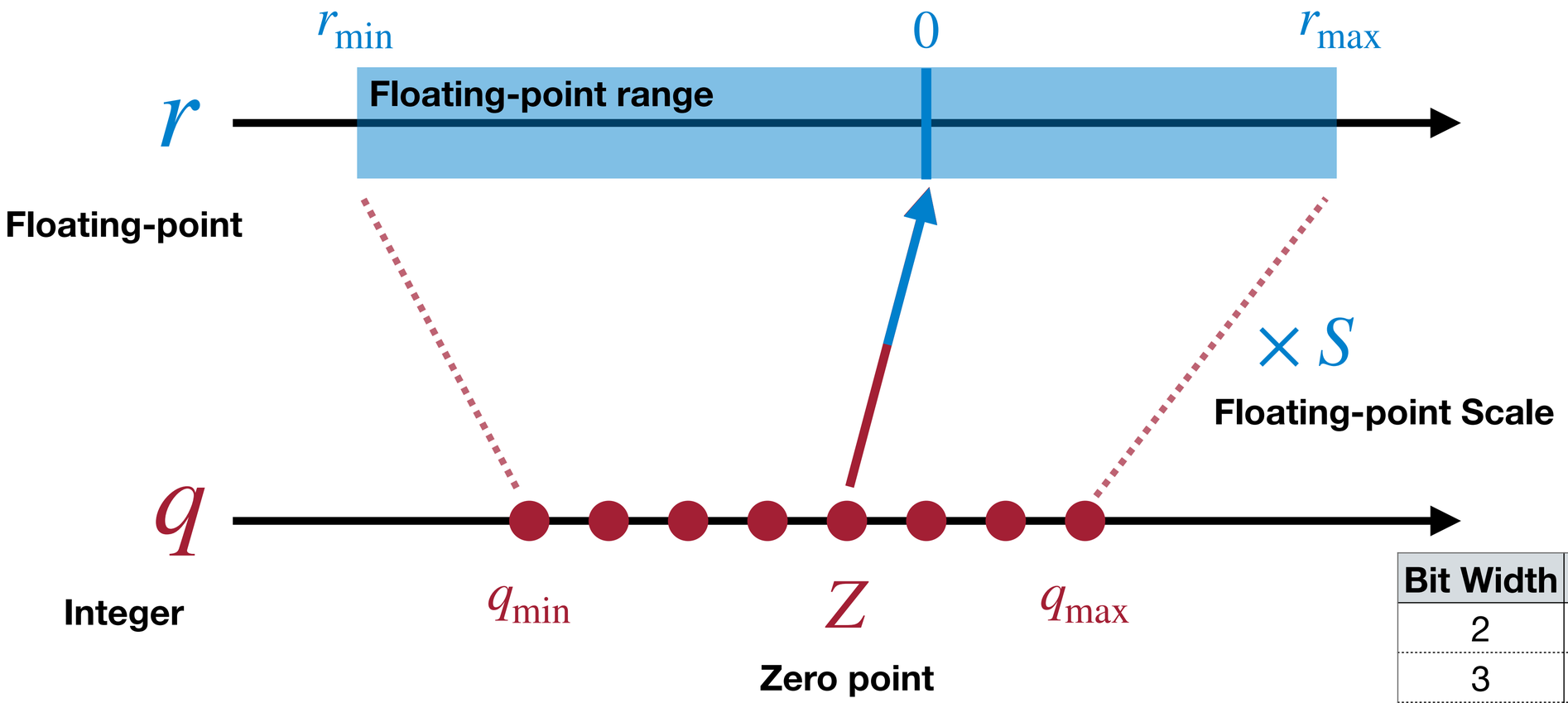
An affine mapping of integers to real numbers $r = S(q - Z)$



Binary	Decimal
01	1
00	0
11	-1
10	-2

Linear Quantization

An affine mapping of integers to real numbers $r = S(q - Z)$



Bit Width	q_{min}	q_{max}
2	-2	1
3	-4	3
4	-8	7
N	-2^{N-1}	$2^{N-1}-1$

Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference [Jacob *et al.*, CVPR 2018]

Linear Quantized Fully-Connected Layer

Linear Quantization is an affine mapping of integers to real numbers $r = S(q - Z)$

- Consider the following fully-connected layer.

$$\mathbf{Y} = \mathbf{W}\mathbf{X} + \mathbf{b}$$

$$Z_{\mathbf{W}} = 0$$

$$Z_{\mathbf{b}} = 0, \quad S_{\mathbf{b}} = S_{\mathbf{W}} S_{\mathbf{X}}$$

$$\mathbf{q}_{bias} = \mathbf{q}_{\mathbf{b}} - Z_{\mathbf{X}} \mathbf{q}_{\mathbf{W}}$$

$$\mathbf{q}_{\mathbf{Y}} = \frac{S_{\mathbf{W}} S_{\mathbf{X}}}{S_{\mathbf{Y}}} \left(\mathbf{q}_{\mathbf{W}} \mathbf{q}_{\mathbf{X}} + \mathbf{q}_{bias} \right) + Z_{\mathbf{Y}}$$

Rescale to *N*-bit Int | **_N_-bit Int Mult. 32-bit Int Add.** | **_N_-bit Int Add**

Note: both $\mathbf{q}_{\mathbf{b}}$ and $\mathbf{q}_{bias}$ are 32 bits.

Linear Quantized Convolution Layer

Linear Quantization is an affine mapping of integers to real numbers $r = S(q - Z)$

- Consider the following convolution layer.

$$\mathbf{Y} = \text{Conv}(\mathbf{W}, \mathbf{X}) + \mathbf{b}$$

$$Z_{\mathbf{W}} = 0$$

$$Z_{\mathbf{b}} = 0, \quad S_{\mathbf{b}} = S_{\mathbf{W}} S_{\mathbf{X}}$$

$$\mathbf{q}_{bias} = \mathbf{q}_{\mathbf{b}} - \text{Conv}(\mathbf{q}_{\mathbf{W}}, Z_{\mathbf{X}})$$

$$\mathbf{q}_{\mathbf{Y}} = \frac{S_{\mathbf{W}} S_{\mathbf{X}}}{S_{\mathbf{Y}}} \left(\text{Conv}(\mathbf{q}_{\mathbf{W}}, \mathbf{q}_{\mathbf{X}}) + \mathbf{q}_{bias} \right) + Z_{\mathbf{Y}}$$

Rescale to *N*-bit Int

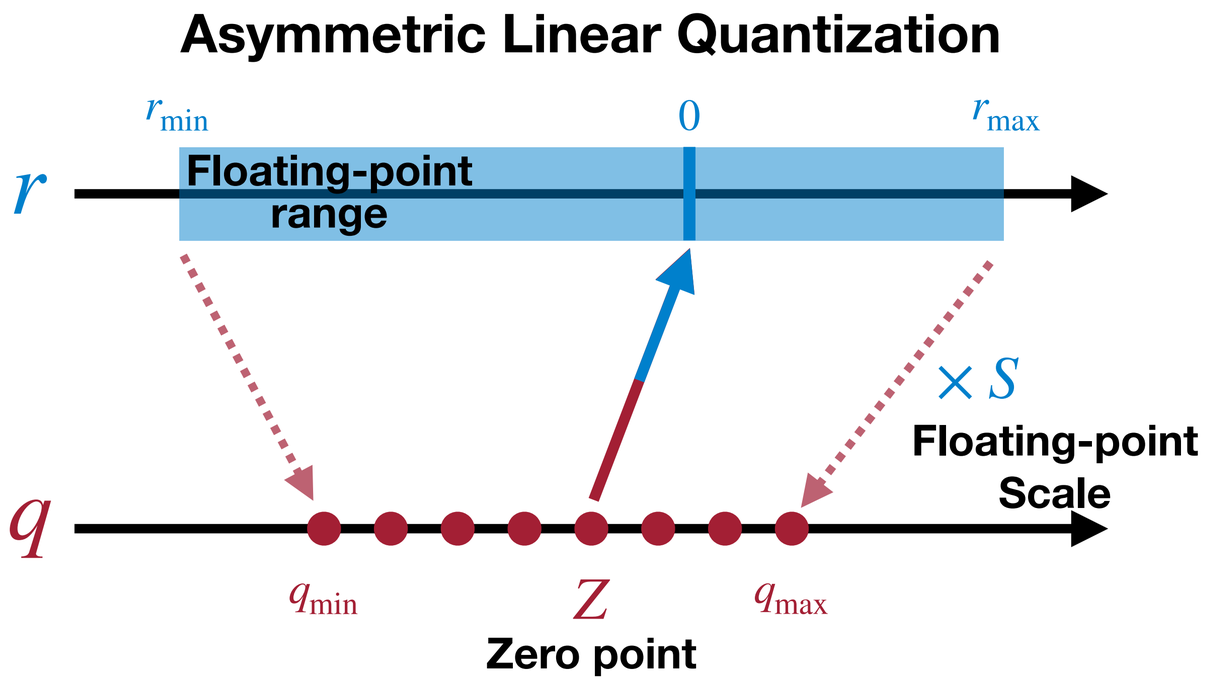
***N*-bit Int Mult. 32-bit Int Add.**

***N*-bit Int Add**

Note: both $\mathbf{q}_{\mathbf{b}}$ and $\mathbf{q}_{bias}$ are 32 bits.

Scale and Zero Point of Linear Quantization

Linear Quantization is an affine mapping of integers to real numbers $r = S(q - Z)$



2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

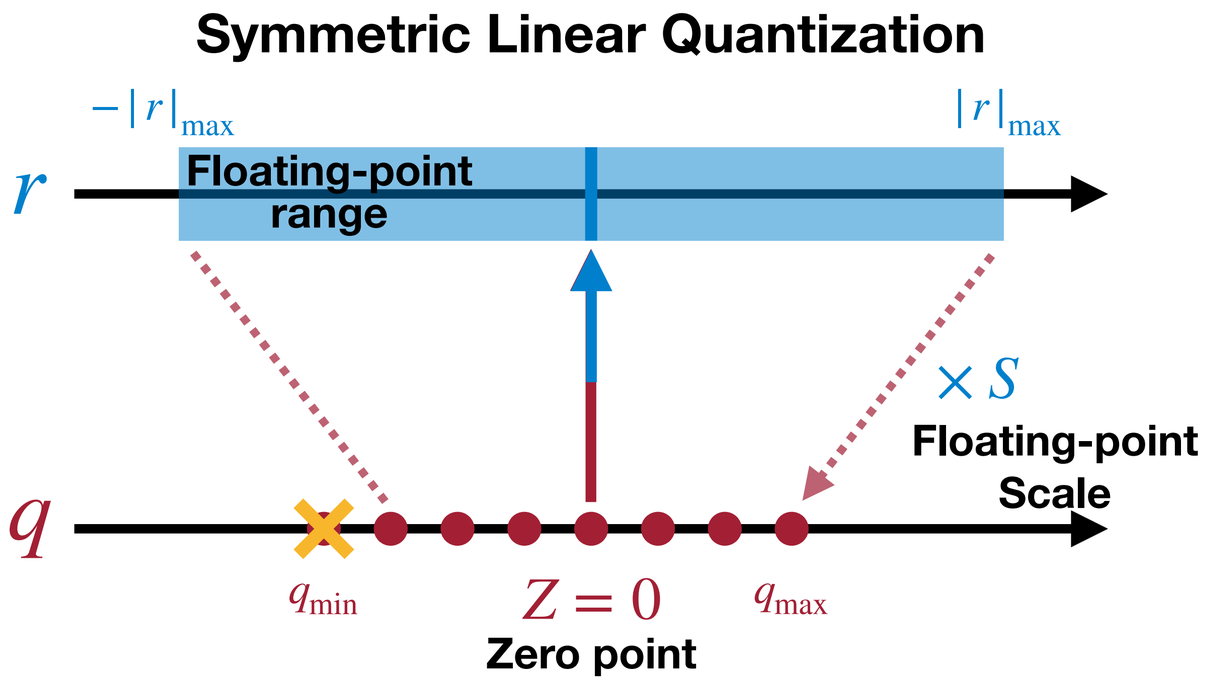
양자화 하는 지식 수식 알고 있어야함 GPT 보고 내 답라고 하기

$$S = \frac{r_{max} - r_{min}}{q_{max} - q_{min}} = \frac{2.12 - (-1.08)}{1 - (-2)} = 1.07$$

$$Z = q_{min} - \frac{r_{min}}{S} = \text{round}(-2 - \frac{-1.08}{1.07}) = -1$$

Scale and Zero Point of Linear Quantization

Linear Quantization is an affine mapping of integers to real numbers $r = S(q - Z)$



2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

$$S = \frac{|r|_{\max}}{q_{\max}} = \frac{2.12}{1} = 2.12 \qquad Z = 0$$

Post-Training Quantization

How should we get the optimal linear quantization parameters (S, Z)?

Topic I: Quantization Granularity
Topic II: Dynamic Range Clipping
Topic III: Rounding

Post-Training Quantization

How should we get the optimal linear quantization parameters (S, Z)?

Topic I: Quantization Granularity

Topic II: Dynamic Range Clipping

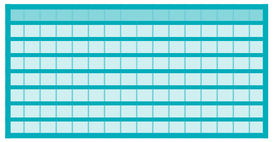
Topic III: Rounding

Quantization Granularity

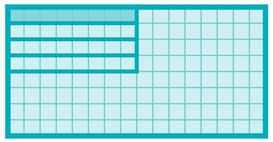
- Per-Tensor Quantization



- Per-Channel Quantization



- Group Quantization



  - Per-Vector Quantization
  - Shared Micro-exponent (MX) data type

Quantization Granularity

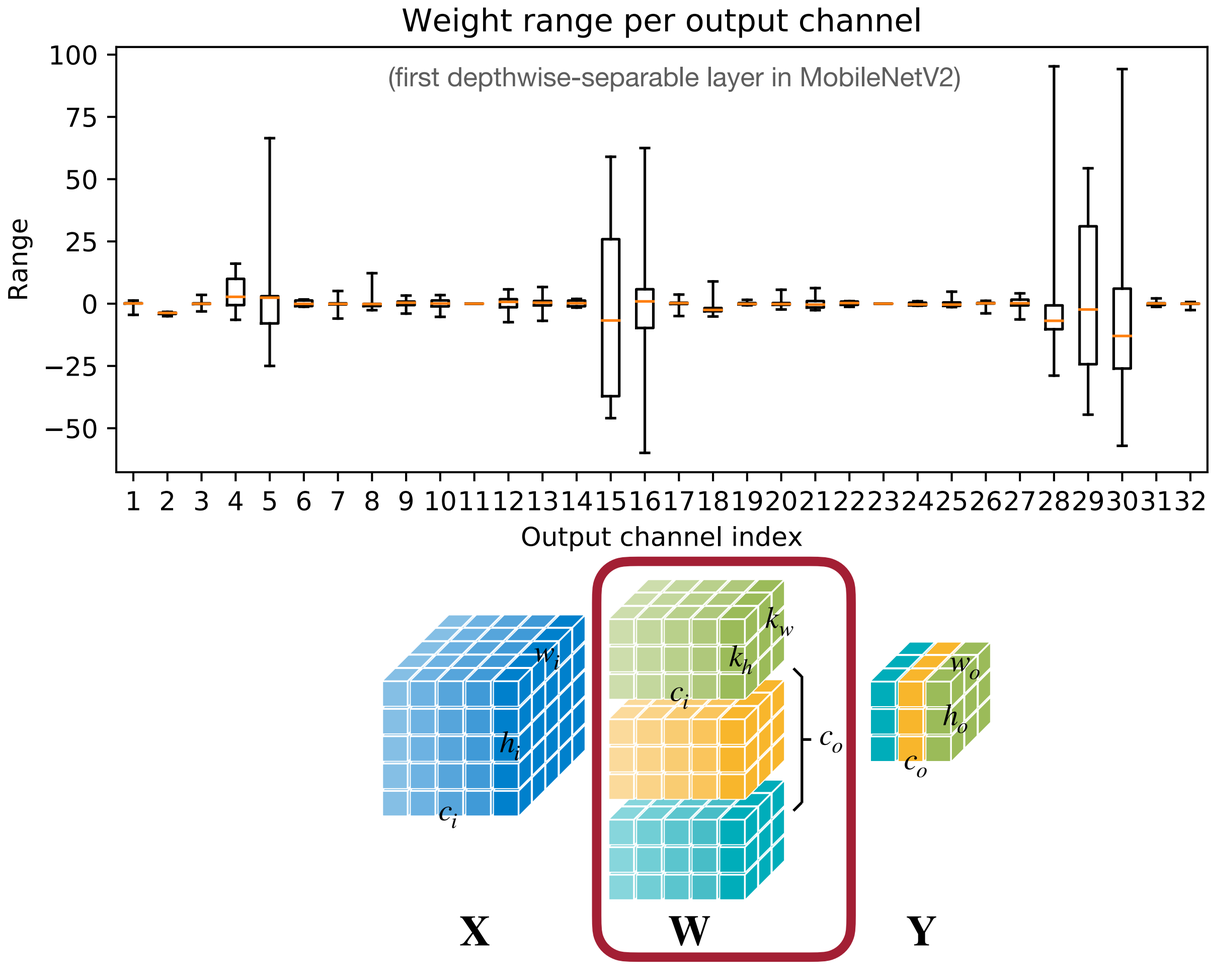
- **Per-Tensor Quantization**



- Per-Channel Quantization

- Group Quantization
  - Per-Vector Quantization
  - Shared Micro-exponent (MX) data type

Symmetric Linear Quantization on Weights



- $|r|_{\max} = |\mathbf{W}|_{\max}$
- Using *single* scale S for whole weight tensor (**Per-Tensor Quantization**)
  - works well for large models
  - accuracy drops for small models
- Common failure results from
  - large differences (more than 100×) in ranges of weights for different output channels — outlier weight
- Solution: **Per-Channel Quantization**

Data-Free Quantization Through Weight Equalization and Bias Correction [Markus *et al.*, ICCV 2019]
Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference [Jacob *et al.*, CVPR 2018]

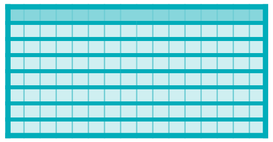
Quantization Granularity

- Per-Tensor Quantization
  = layer Quantization



하나의 layer 전체, mxn 전체를 본다

- **Per-Channel Quantization**



≒ Structured pruning

- Group Quantization
  - Per-Vector Quantization
  - Shared Micro-exponent (MX) data type

정밀도를 더 세분화하게
나눌수록 성능이 좋을 수 밖에
없다

⇒ but Cost ↑

Pytorch에서 학습시킬 때
그래서 성능 good
실용에선 per-tensor
쓴다

Per-Channel Weight Quantization

Example: 2-bit linear quantization

ic

oc				
	2.09	-0.98	1.48	0.09
	0.05	-0.14	-1.08	2.12
	-0.91	1.92	0	-1.03
	1.87	0	1.53	1.49

Per-Channel Quantization

Per-Tensor Quantization

Per-Channel Weight Quantization

Example: 2-bit linear quantization

Per-Channel Quantization

oc \ ic				
	2.09	-0.98	1.48	0.09
	0.05	-0.14	-1.08	2.12
	-0.91	1.92	0	-1.03
	1.87	0	1.53	1.49

Per-Tensor Quantization

$$|r|_{\max} = 2.12$$

$$S = \frac{|r|_{\max}}{q_{\max}} = \frac{2.12}{2^{2-1} - 1} = 2.12$$

1	0	1	0
0	0	-1	1
0	1	0	0
1	0	1	1

Quantized

2.12	0	2.12	0
0	0	-2.12	2.12
0	2.12	0	0
2.12	0	2.12	2.12

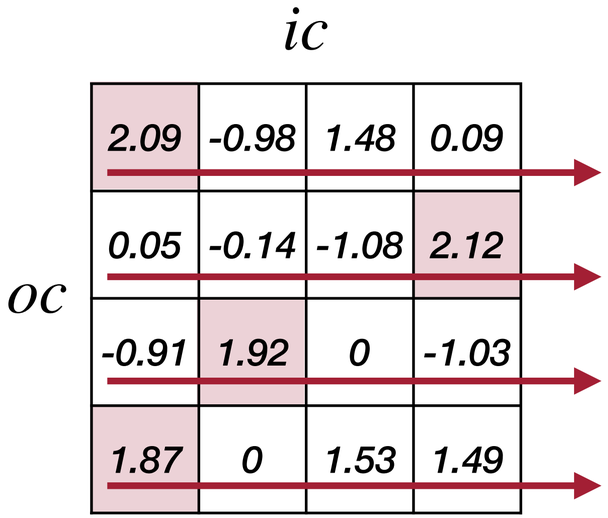
Reconstructed

Quantization error

$$\|\mathbf{W} - S\mathbf{q}_{\mathbf{W}}\|_F = 2.28$$

Per-Channel Weight Quantization

Example: 2-bit linear quantization



Per-Channel Quantization

$$|r|_{\max} = 2.09 \qquad S_0 = 2.09$$

$$|r|_{\max} = 2.12 \qquad S_1 = 2.12$$

$$|r|_{\max} = 1.92 \qquad S_2 = 1.92$$

$$|r|_{\max} = 1.87 \qquad S_3 = 1.87$$

Per-Tensor Quantization

$$|r|_{\max} = 2.12$$

$$S = \frac{|r|_{\max}}{q_{\max}} = \frac{2.12}{2^{2-1} - 1} = 2.12$$

1	0	1	0
0	0	-1	1
0	1	0	0
1	0	1	1

Quantized

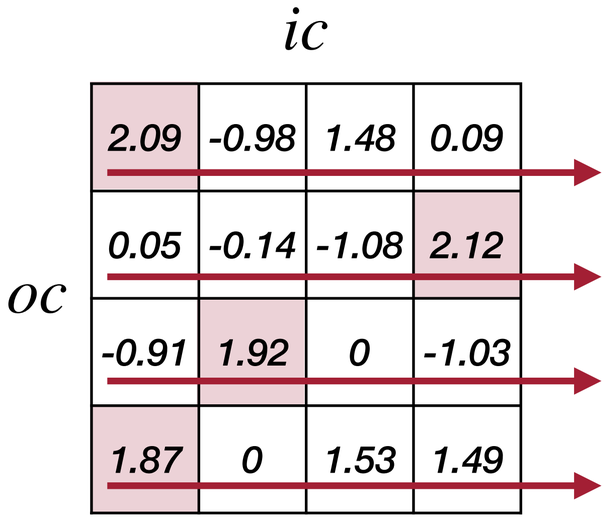
2.12	0	2.12	0
0	0	-2.12	2.12
0	2.12	0	0
2.12	0	2.12	2.12

Reconstructed

$$\|\mathbf{W} - S\mathbf{q}_{\mathbf{W}}\|_F = 2.28$$

Per-Channel Weight Quantization

Example: 2-bit linear quantization



Per-Channel Quantization

$|r|_{\max} = 2.09 \quad S_0 = 2.09$

$|r|_{\max} = 2.12 \quad S_1 = 2.12$

$|r|_{\max} = 1.92 \quad S_2 = 1.92$

$|r|_{\max} = 1.87 \quad S_3 = 1.87$

1	0	1	0
0	0	-1	1
0	1	0	-1
1	0	1	1

Quantized

2.09	0	2.09	0
0	0	-2.12	2.12
0	1.92	0	-1.92
1.87	0	1.87	1.87

Reconstructed

$\|\mathbf{W} - \mathbf{S} \odot \mathbf{q_W}\|_F = 2.08$

Per-Tensor Quantization

$|r|_{\max} = 2.12$

$S = \frac{|r|_{\max}}{q_{\max}} = \frac{2.12}{2^{2-1} - 1} = 2.12$

1	0	1	0
0	0	-1	1
0	1	0	0
1	0	1	1

Quantized

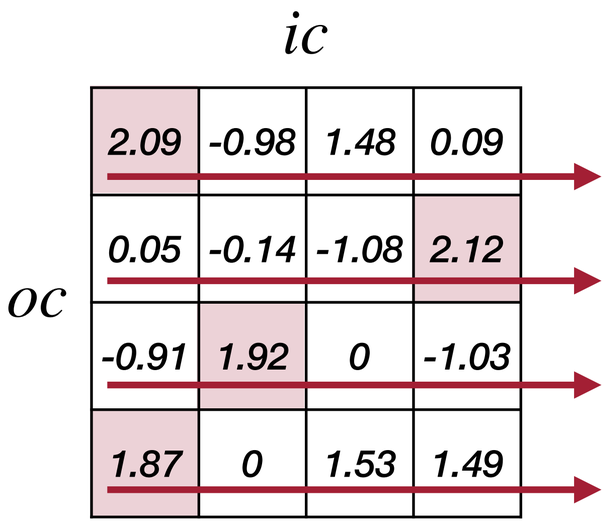
2.12	0	2.12	0
0	0	-2.12	2.12
0	2.12	0	0
2.12	0	2.12	2.12

Reconstructed

$\|\mathbf{W} - S\mathbf{q_W}\|_F = 2.28$

Per-Channel Weight Quantization

Example: 2-bit linear quantization



Per-Channel Quantization

$|r|_{\max} = 2.09 \quad S_0 = 2.09$

$|r|_{\max} = 2.12 \quad S_1 = 2.12$

$|r|_{\max} = 1.92 \quad S_2 = 1.92$

$|r|_{\max} = 1.87 \quad S_3 = 1.87$

1	0	1	0
0	0	-1	1
0	1	0	-1
1	0	1	1

Quantized

2.09	0	2.09	0
0	0	-2.12	2.12
0	1.92	0	-1.92
1.87	0	1.87	1.87

Reconstructed

$\|\mathbf{W} - \mathbf{S} \odot \mathbf{q}_\mathbf{W}\|_F = 2.08$

<

Per-Tensor Quantization

$|r|_{\max} = 2.12$

$S = \frac{|r|_{\max}}{q_{\max}} = \frac{2.12}{2^{2-1} - 1} = 2.12$

1	0	1	0
0	0	-1	1
0	1	0	0
1	0	1	1

Quantized

2.12	0	2.12	0
0	0	-2.12	2.12
0	2.12	0	0
2.12	0	2.12	2.12

Reconstructed

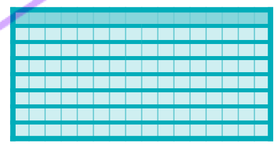
$\|\mathbf{W} - S\mathbf{q}_\mathbf{W}\|_F = 2.28$

Quantization Granularity

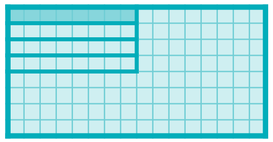
- Per-Tensor Quantization



- Per-Channel Quantization



- **Group Quantization**
  - **Per-Vector Quantization**
  - **Shared Micro-exponent (MX) data type**

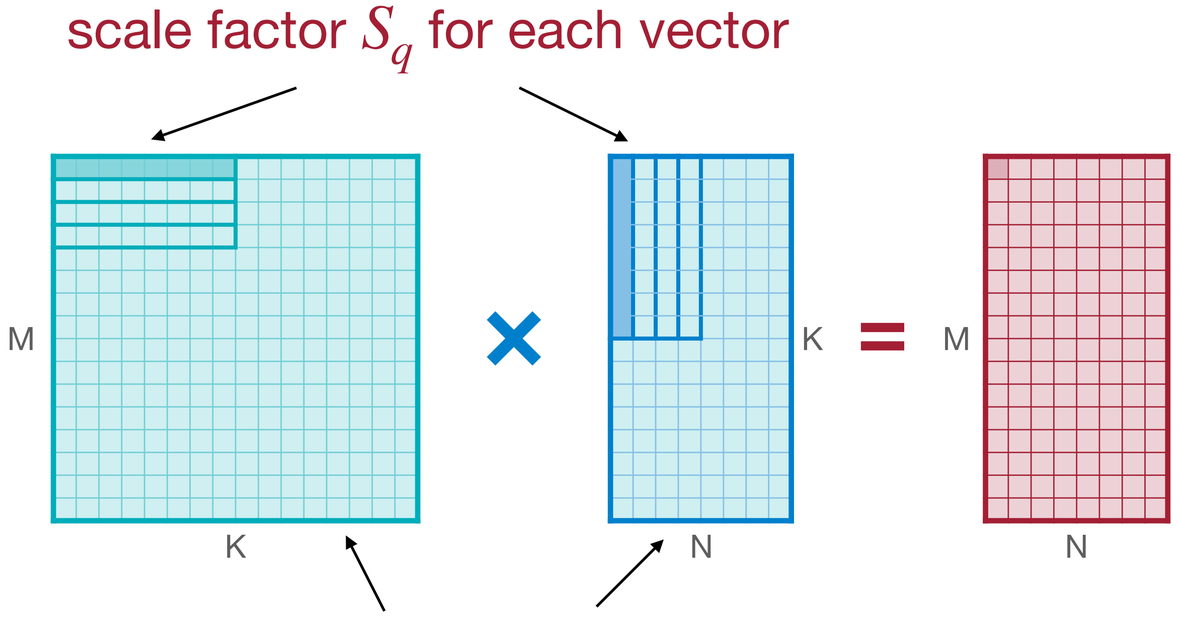


VS-Quant: Per-vector Scaled Quantization

Hierarchical scaling factor

- $r = S(q - Z) \rightarrow r = \gamma \cdot S_q(q - Z)$
  - γ is a floating-point coarse grained scale factor
  - S_q is an integer per-vector scale factor
  - achieves a balance between accuracy and hardware efficiency by
    - less expensive integer scale factors at finer granularity
    - more expensive floating-point scale factors at coarser granularity

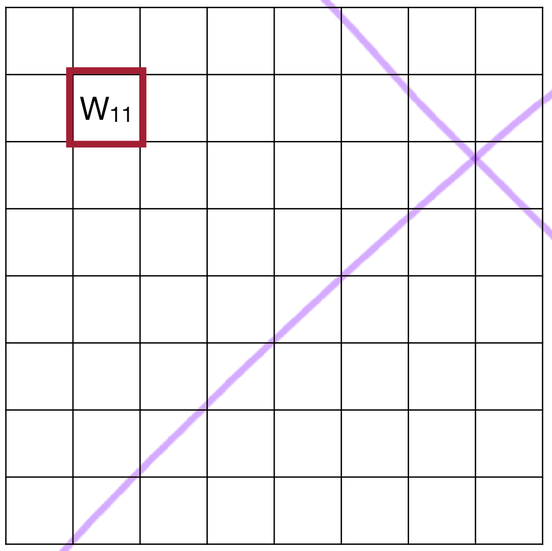
- Memory Overhead of two-level scaling:
  - Given 4-bit quantization with 4-bit per-vector scale for every 16 elements, the effective bit width is 4 + 4 / 16 = 4.25 bits.



VS-Quant: Per-Vector Scaled Quantization for Accurate Low-Precision Neural Network Inference [Steve Dai, *et al.*]

Group Quantization

Multi-level scaling scheme



$$r = (q - z) \cdot s \rightarrow$$
$$r = (q - z) \cdot s_{l_0} \cdot s_{l_1} \cdot \cdots$$

r : real number value

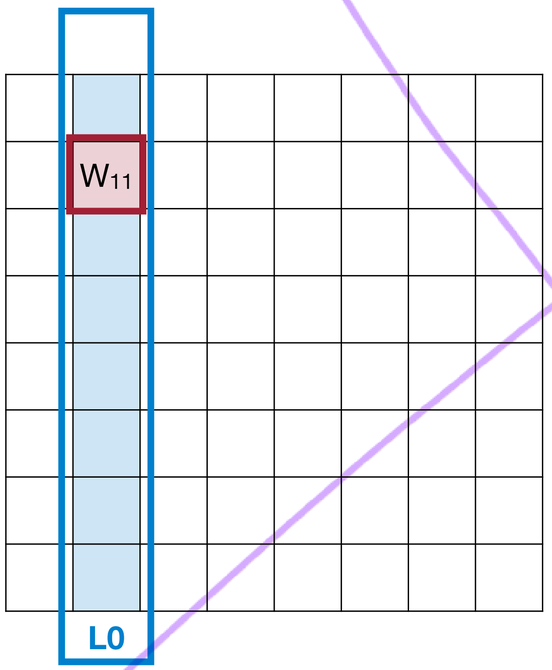
q : quantized value

z : zero point ($z = 0$ is symmetric quantization)

s : scale factors of different levels

Group Quantization

Multi-level scaling scheme



$$r = (q - z) \cdot s_{l_0} \cdot s_{l_1} \cdot \cdots$$

r : real number value

q : quantized value

z : zero point ($z = 0$ is symmetric quantization)

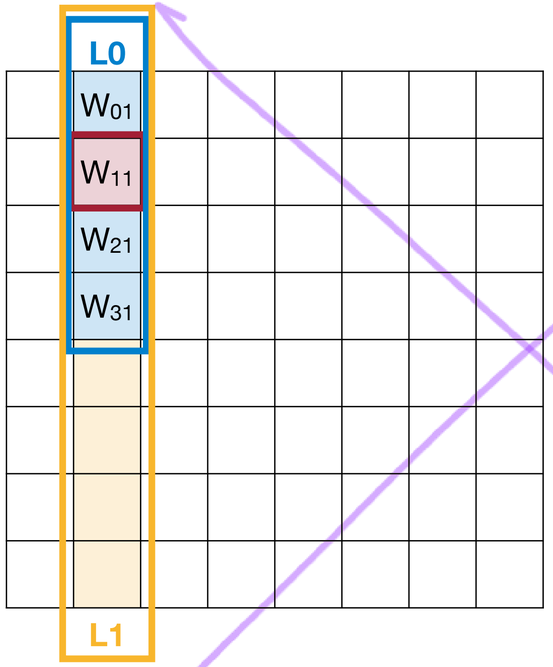
s : scale factors of different levels

FP16 s_{l_0} INT4 q

Quantization Approach	Data Type	L0 Group Size	L0 Scale Data Type	L1 Group Size	L1 Scale Data Type	Effective Bit Width
Per-Channel Quant	INT4	Per Channel	FP16	-	-	4

Group Quantization

Multi-level scaling scheme



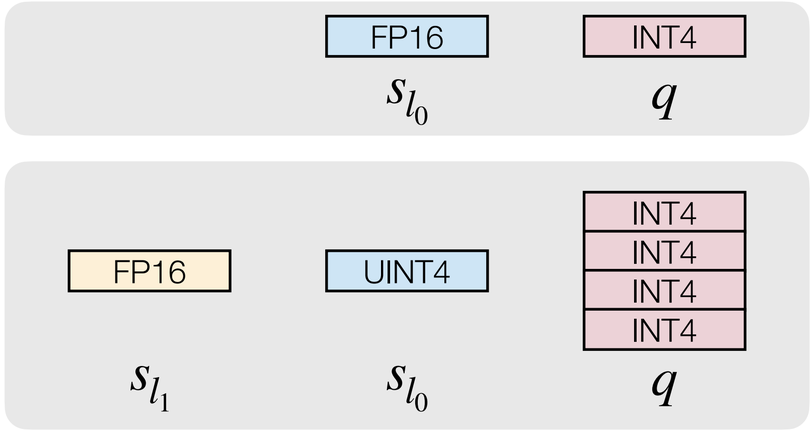
$$r = (q - z) \cdot s_{l_0} \cdot s_{l_1} \cdot \cdots$$

r : real number value

q : quantized value

z : zero point ($z = 0$ is symmetric quantization)

s : scale factors of different levels

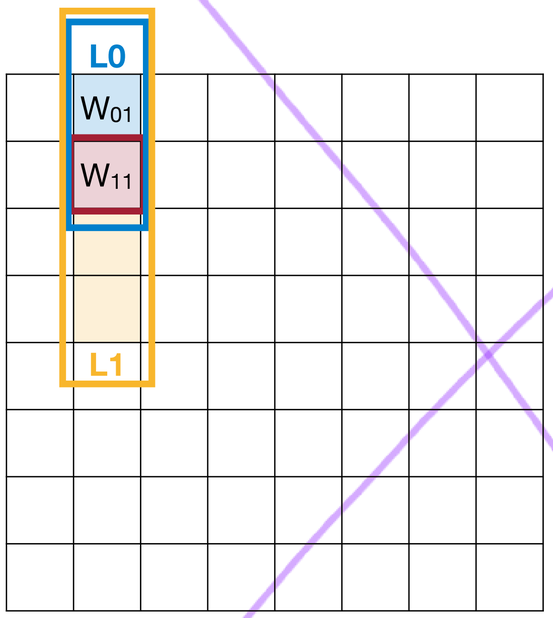


Quantization Approach	Data Type	L0 Group Size	L0 Scale Data Type	L1 Group Size	L1 Scale Data Type	Effective Bit Width
Per-Channel Quant	INT4	Per Channel	FP16	-	-	4
VSQ	INT4	16	UINT4	Per Channel	FP16	4+4/16=4.25

VS-Quant: Per-Vector Scaled Quantization for Accurate Low-Precision Neural Network Inference [Steve Dai, *et al.*]

Group Quantization

Multi-level scaling scheme



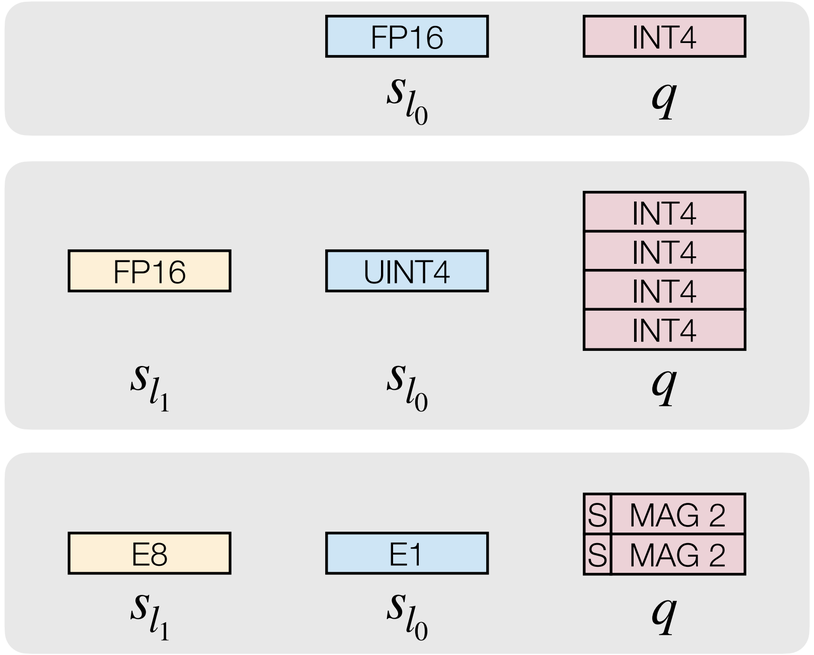
$$r = (q - z) \cdot s_{l_0} \cdot s_{l_1} \cdot \cdots$$

r : real number value

q : quantized value

z : zero point ($z = 0$ is symmetric quantization)

s : scale factors of different levels



Quantization Approach	Data Type	L0 Group Size	L0 Scale Data Type	L1 Group Size	L1 Scale Data Type	Effective Bit Width
Per-Channel Quant	INT4	Per Channel	FP16	-	-	4
VSQ	INT4	16	UINT4	Per Channel	FP16	4+4/16=4.25
MX4	S1M2	2	E1M0	16	E8M0	3+1/2+8/16=4
MX6	S1M4	2	E1M0	16	E8M0	5+1/2+8/16=6
MX9	S1M7	2	E1M0	16	E8M0	8+1/2+8/16=9

With Shared Microexponents, A Little Shifting Goes a Long Way [Bita Rouhani *et al.*]

Post-Training Quantization

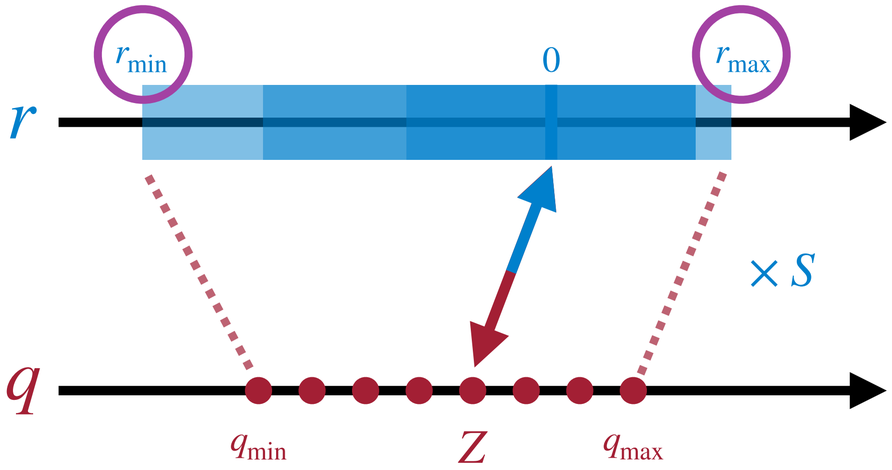
How should we get the optimal linear quantization parameters (S, Z)?

Topic I: Quantization Granularity

Topic II: Dynamic Range Clipping

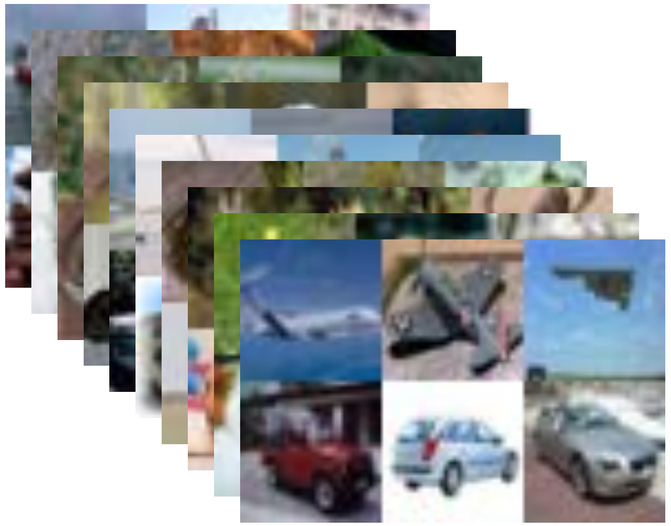
Topic III: Rounding

Linear Quantization on Activations



- Unlike weights, the activation range varies across inputs.
- To determine the floating-point range, the activations statistics are gathered ***before*** deploying the model.

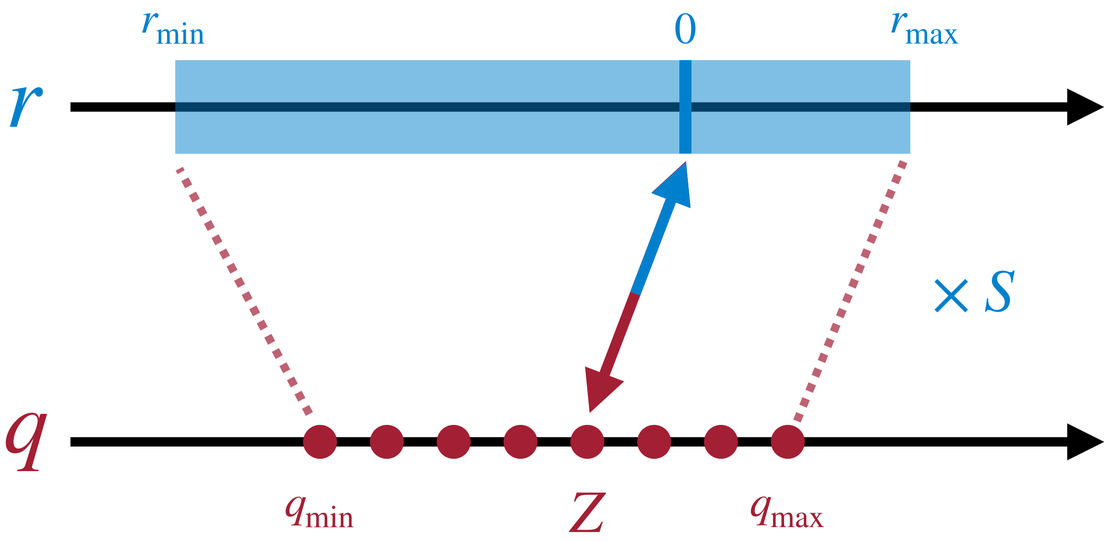
min max range



Dynamic Range for Activation Quantization

Collect activations statistics before deploying the model

$$\hat{r}^{(t)}_{\max,\min} = \alpha \cdot r^{(t)}_{\max,\min} + (1-\alpha) \cdot \hat{r}^{(t-1)}_{\max,\min}$$



- Type 1: During training 잘 안 씀
  - Exponential moving averages (EMA)
    - observed ranges are smoothed across thousands of training steps

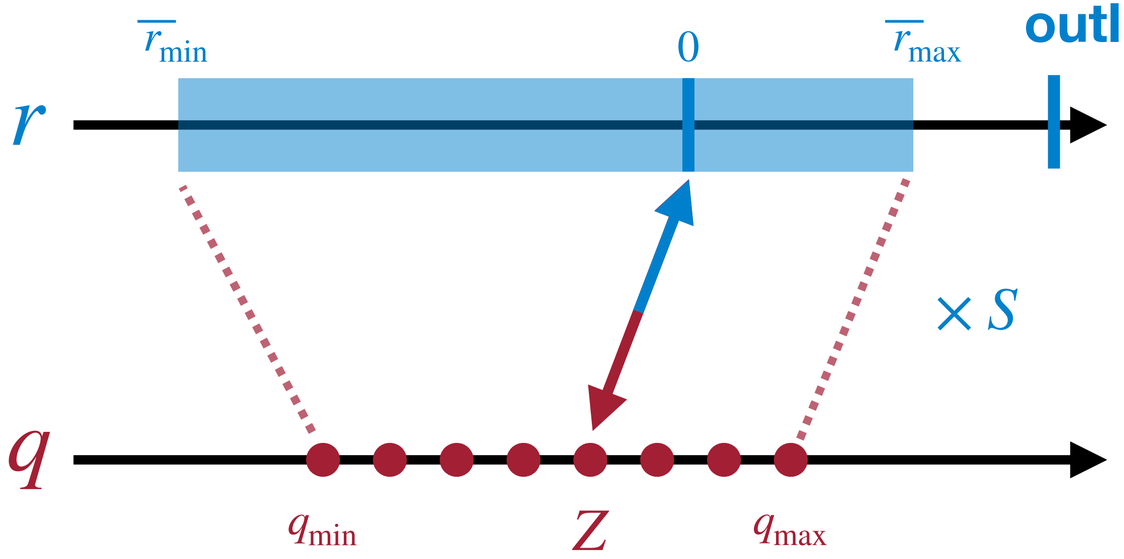
기울기 큰 가중치

새로 들어오는 값 낮은 가중치

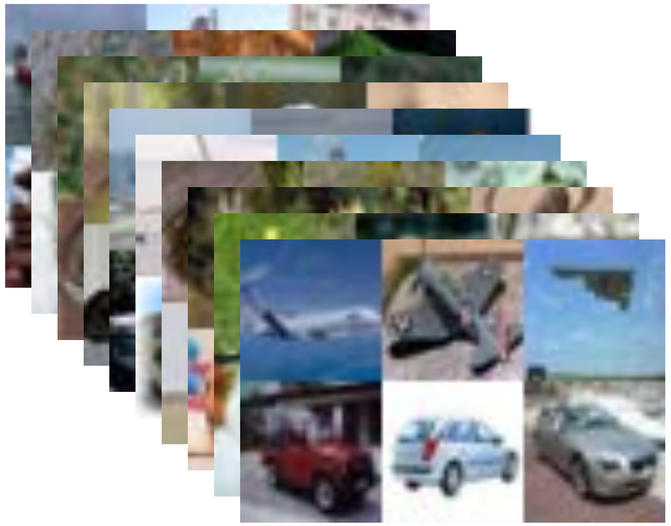
Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference [Jacob *et al.*, CVPR 2018]

Dynamic Range for Activation Quantization

Collect activations statistics before deploying the model



- Type 2: By running a few "calibration" batches of samples on the trained FP32 model
  - spending dynamic range on the outliers hurts the representation ability.
  - use *mean* of the min/max of each sample in the batches
  - analytical calculation (see next slide)

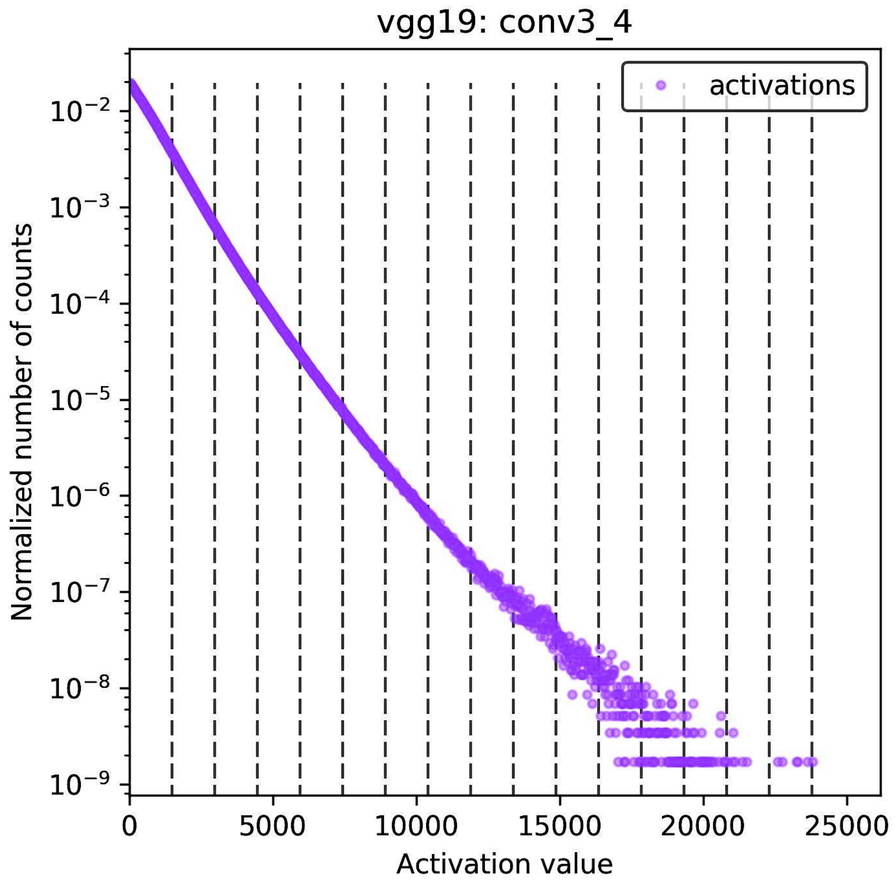


Neural Network Distiller

Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference [Jacob *et al.*, CVPR 2018]

Dynamic Range for Activation Quantization

Collect activations statistics before deploying the model



- Type 2: By running a few "calibration" batches of samples on the trained FP32 model
  - *minimize loss of information*, since integer model encodes the same information as the original floating-point model.
    - loss of information is measured by Kullback-Leibler divergence (relative entropy or information divergence):
      - for two discrete probability distributions P, Q

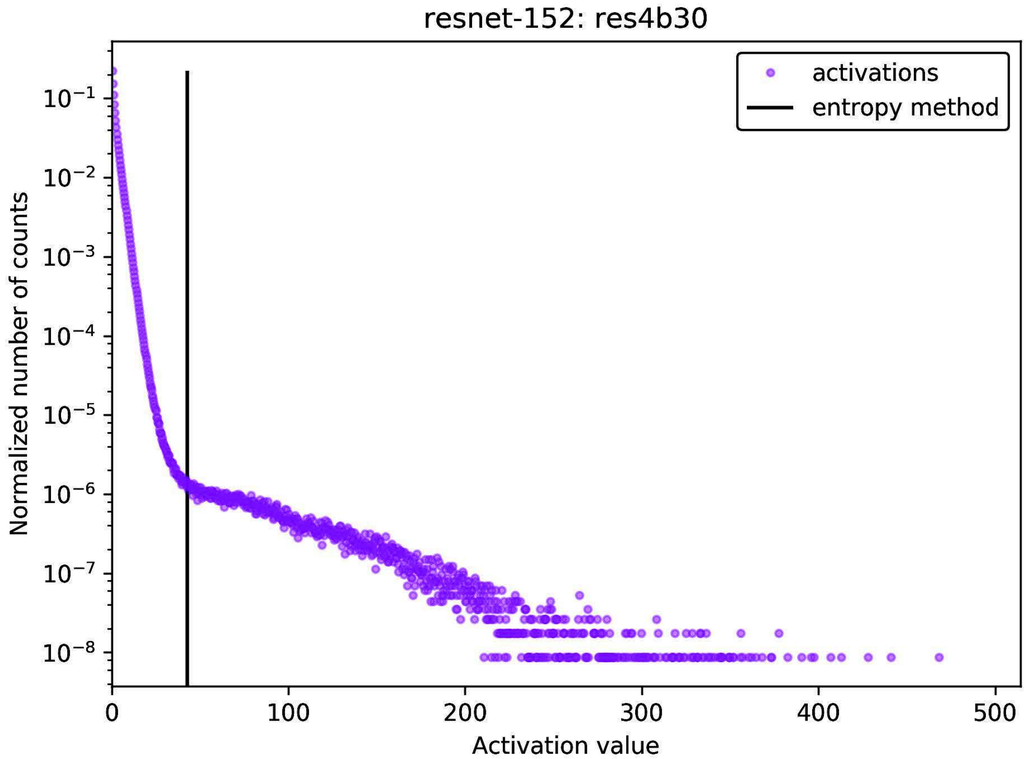
$$D_{KL}(P\|Q) = \sum_{i}^{N} P(x_i)\log\frac{P(x_i)}{Q(x_i)}$$

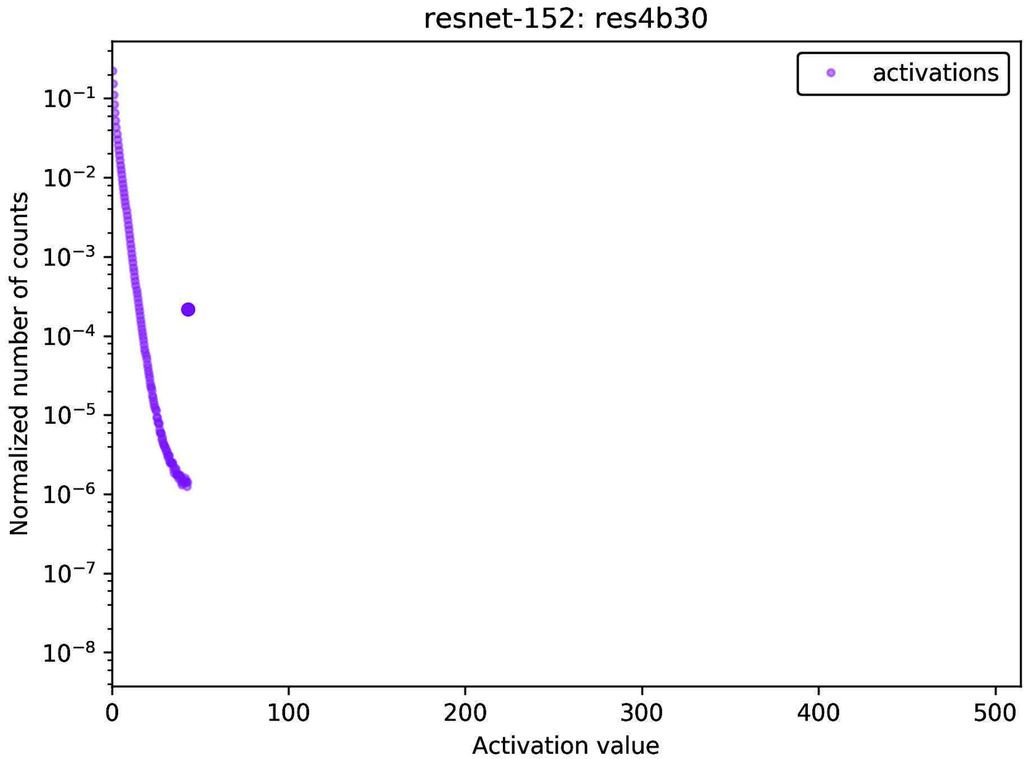
      - intuition: KL divergence measures the amount of information lost when approximating a given encoding.

8-bit Inference with TensorRT [Szymon Migacz, 2017]

Dynamic Range for Activation Quantization

Minimize loss of information by minimizing the KL divergence



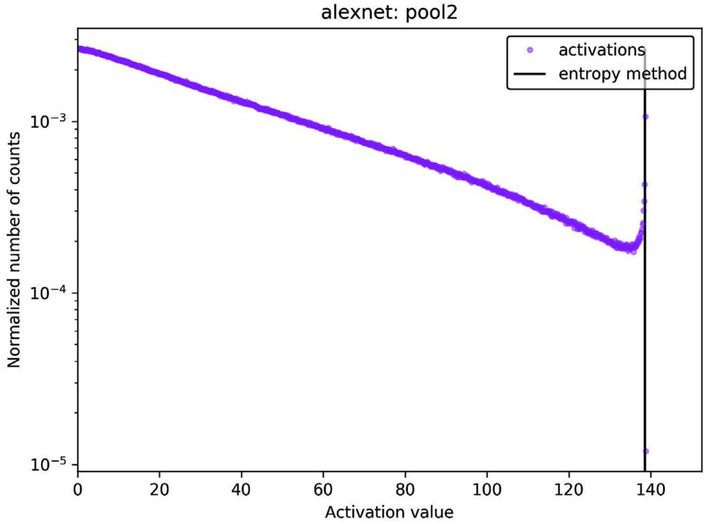


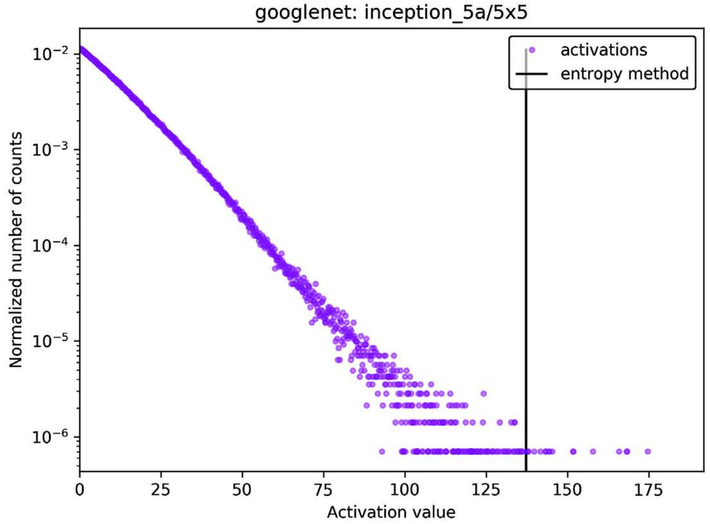
8-bit Inference with TensorRT [Szymon Migacz, 2017]

Dynamic Range for Activation Quantization

Minimize loss of information by minimizing the KL divergence

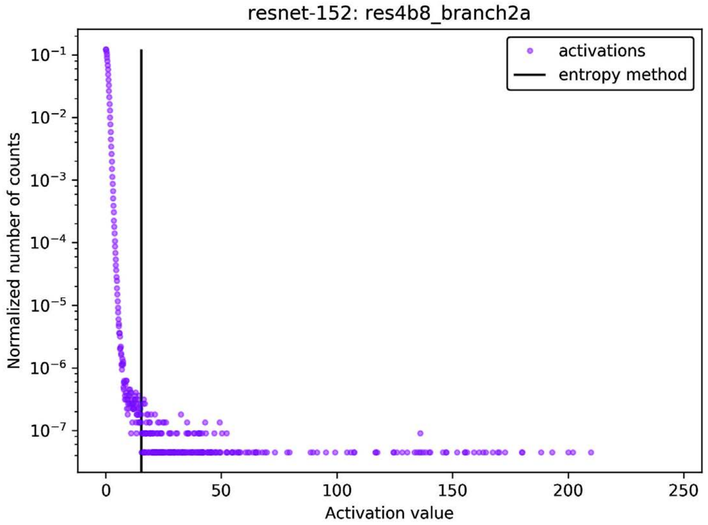
AlexNet: Pool 2

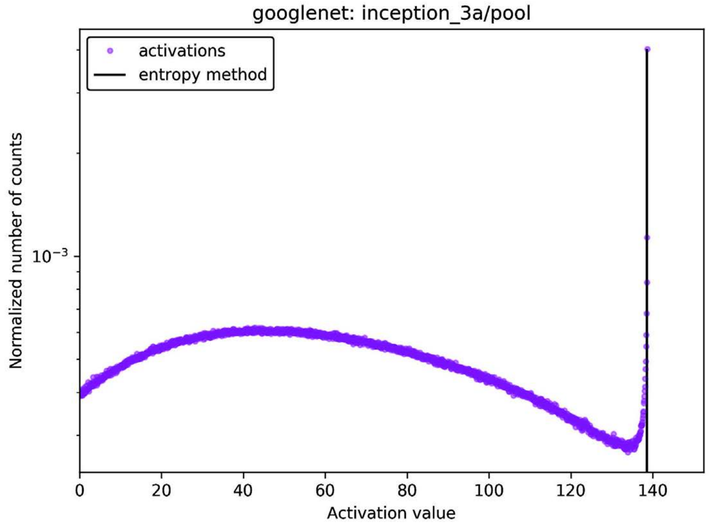




GoogleNet: incpetion_5a/5x5

ResNet-152: res4b8_branch2a





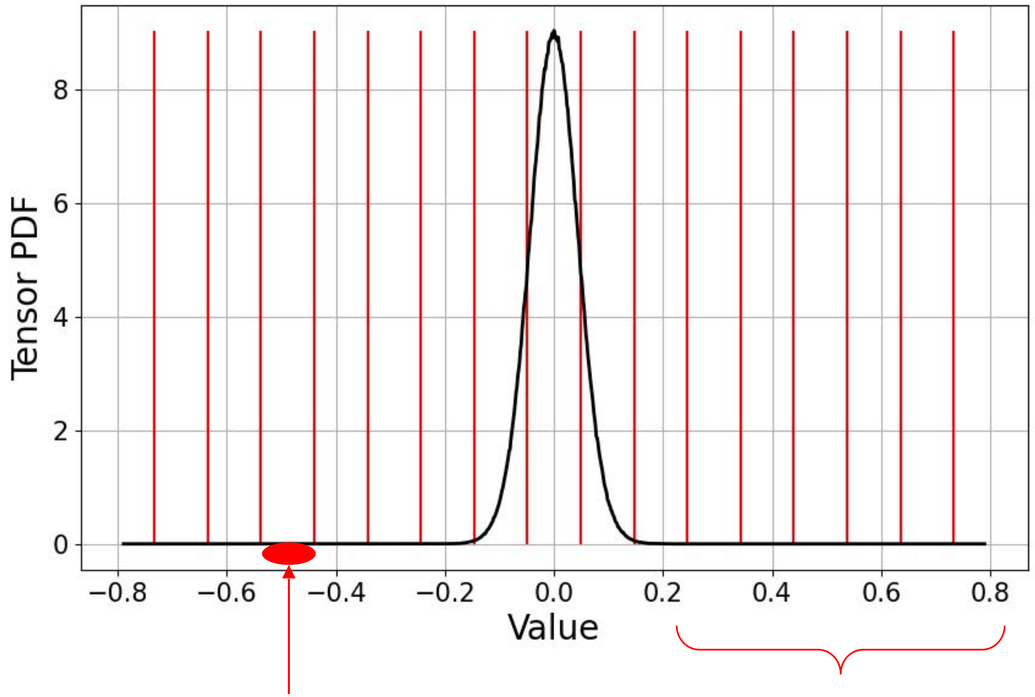
GoogleNet: incpetion_3a/pool

8-bit Inference with TensorRT [Szymon Migacz, 2017]

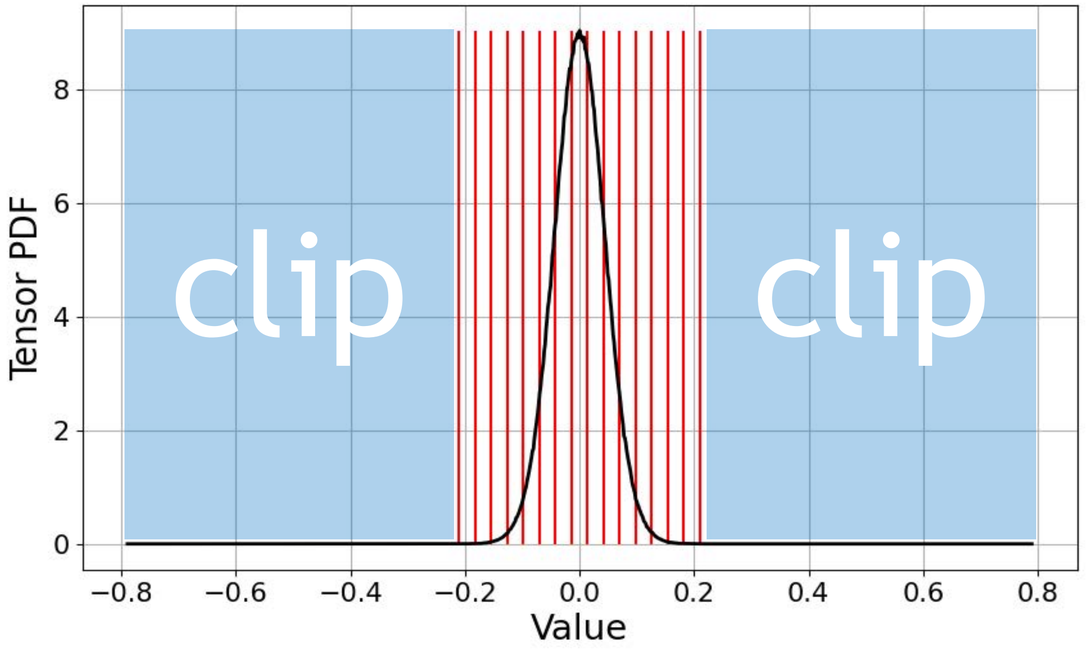
Dynamic Range for Quantization

Minimize mean-square-error (MSE) using Newton-Raphson method

max-scaled quantization



clipped quantization

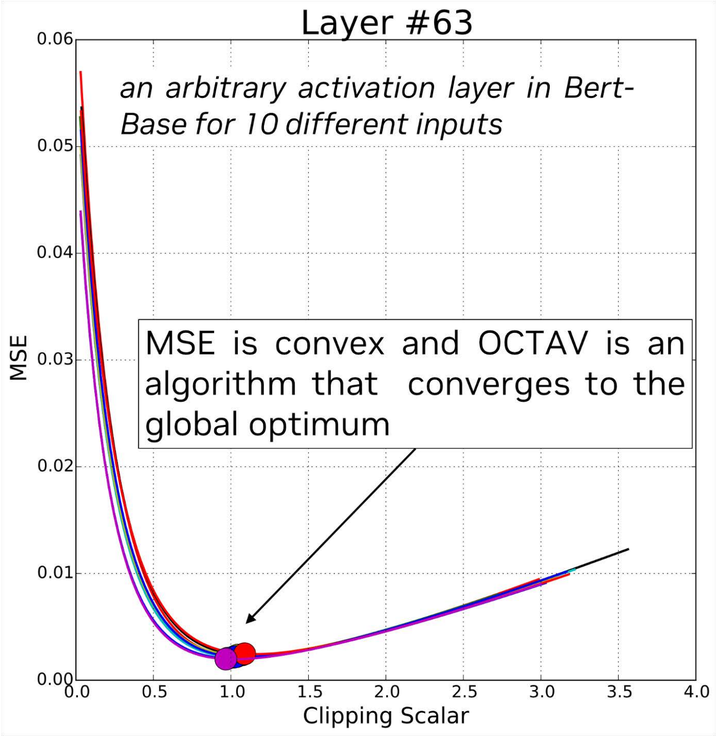


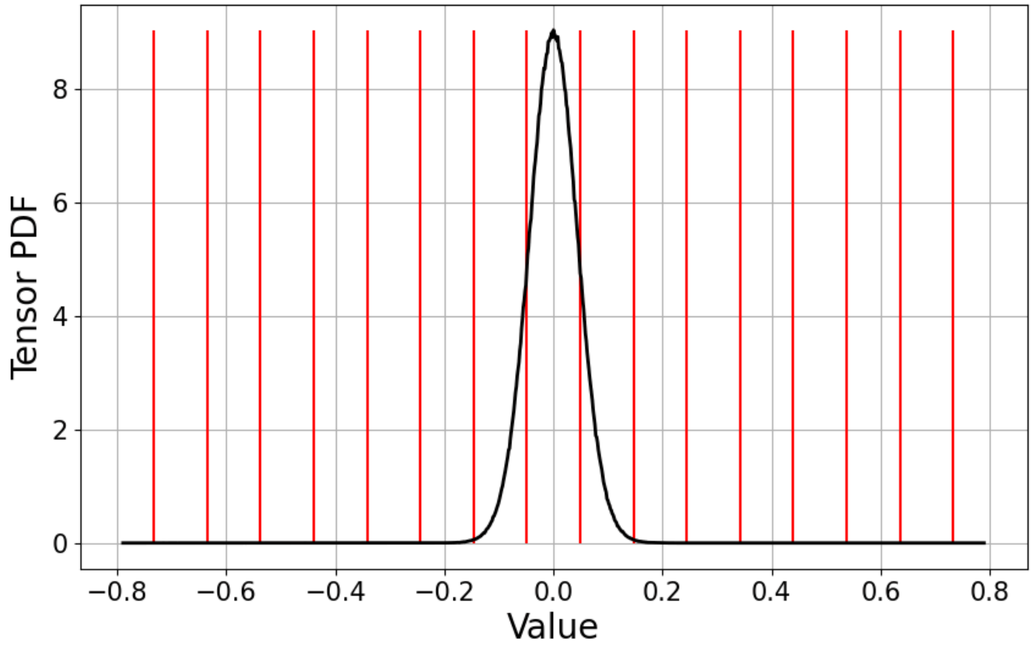
Quantization error 최소가 되는 min max 찾는다

Optimal Clipping and Magnitude-aware Differentiation for Improved Quantization-aware Training [Sakr *et al.*, ICML 2022]

Dynamic Range for Quantization

Minimize mean-square-error (MSE) using Newton-Raphson method





Network	FP32 Accuracy	OCTAV int4
ResNet-50	76.07	75.84
MobileNet-V2	71.71	70.88
Bert-Large	91.00	87.09

Optimal Clipping and Magnitude-aware Differentiation for Improved Quantization-aware Training [Sakr *et al.*, ICML 2022]

QAT의 Loss Function
⇒ 풀고자하는 task의 loss가 들어감

Post-Training Quantization ⇒ gradient 계산 X

How should we get the optimal linear quantization parameters (S, Z)?

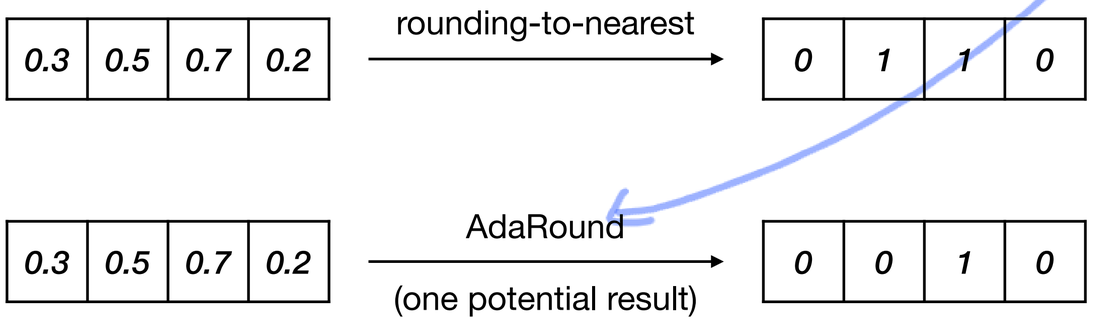
range 결정

Topic I: Quantization Granularity
Topic II: Dynamic Range Clipping
Topic III: Rounding

Adaptive Rounding for Weight Quantization

Rounding-to-nearest is not optimal

- **Philosophy**
  - Rounding-to-nearest is not optimal
  - Weights are correlated with each other. The best rounding for each weight (to nearest) is not the best rounding for the whole tensor



- What is optimal? Rounding that reconstructs the original <u>activation</u> the best, which may be very different
  - For weight quantization only
  - With short-term tuning, (almost) post-training quantization

Up or Down? Adaptive Rounding for Post-Training Quantization [Nagel *et al.*, PMLR 2020]

Adaptive Rounding for Weight Quantization

Rounding-to-nearest is not optimal

- **Method**:
  - Instead of $\lfloor w \rceil$, we want to choose from $\{\lfloor w \rfloor, \lceil w \rceil\}$ to get the best reconstruction
  - We took a learning-based method to find quantized value $\tilde{w} = \lfloor \lfloor w \rfloor + \delta \rceil, \delta \in [0,1]$



Up or Down? Adaptive Rounding for Post-Training Quantization [Nagel *et al.*, PMLR 2020]

Adaptive Rounding for Weight Quantization

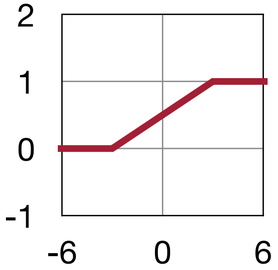
Rounding-to-nearest is not optimal

- **Method**:
  - Instead of $\lfloor w \rfloor$, we want to choose from $\{\lfloor w \rfloor, \lceil w \rceil\}$ to get the best reconstruction
  - We took a learning-based method to find quantized value $\tilde{w} = \lfloor \lfloor w \rfloor + \delta \rceil, \delta \in [0,1]$
  - We optimize the following equation (omit the derivation):

$$\text{argmin}_{\mathbf{V}} \|\mathbf{W}\mathbf{x} - \tilde{\mathbf{W}}\mathbf{x}\|_F^2 + \lambda f_{reg}(\mathbf{V})$$

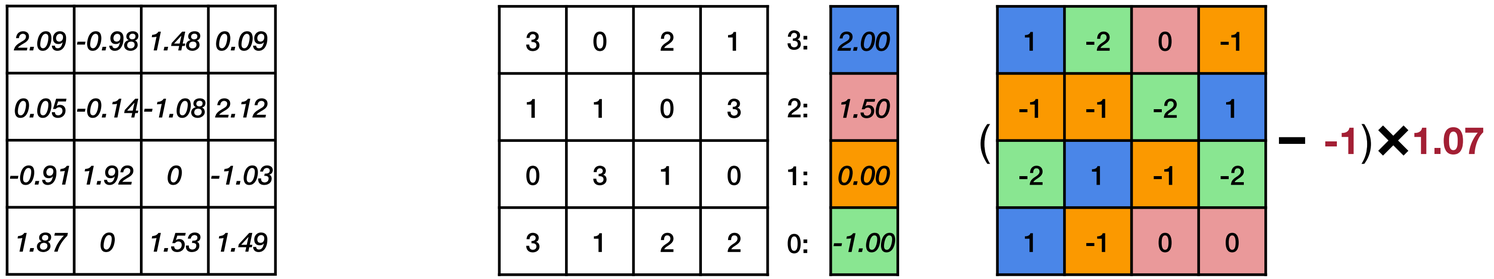
$$\rightarrow \text{argmin}_{\mathbf{V}} \|\mathbf{W}\mathbf{x} - \lfloor \lfloor \mathbf{W} \rfloor + \mathbf{h}(\mathbf{V}) \rceil \mathbf{x}\|_F^2 + \lambda f_{reg}(\mathbf{V})$$

  - $\mathbf{x}$ is the input to the layer, $\mathbf{V}$ is a random variable of the same shape
  - $\mathbf{h}()$ is a function to map the range to $(0,1)$, such as rectified sigmoid
  - $f_{reg}(\mathbf{V})$ is a regularization that encourages $\mathbf{h}(\mathbf{V})$ to be binary
    - $f_{reg}(\mathbf{V}) = \sum_{i,j} 1 - |2h(\mathbf{V}_{i,j}) - 1|^{\beta}$



Up or Down? Adaptive Rounding for Post-Training Quantization [Nagel *et al.*, PMLR 2020]

Neural Network Quantization



		K-Means-based Quantization	Linear Quantization
Storage	Floating-Point Weights	Integer Weights; Floating-Point Codebook	Integer Weights
Computation	Floating-Point Arithmetic	Floating-Point Arithmetic	Integer Arithmetic

- **Zero Point**
  - Asymmetric
  - Symmetric
- **Scaling Granularity**
  - Per-Tensor
  - Per-Channel
  - Group Quantization
- **Range Clipping**
  - Exponential Moving Average
  - Minimizing KL Divergence
  - Minimizing Mean-Square-Error
- **Rounding**
  - Round-to-Nearest
  - AdaRound

Post-Training INT8 Linear Quantization

Activation		Symmetric	Asymmertric
		Per-Tensor	Per-Tensor
		Minimize KL-Divergence	Exponential Moving Average (EMA)
Weight		Symmetric	Symmetric
		Per-Tensor	Per-Channel
Neural Network	GoogleNet	-0.45%	0%
	ResNet-50	-0.13%	-0.6%
	ResNet-152	-0.08%	-1.8%
	MobileNetV1	-	-11.8%
	MobileNetV2	-	-2.1%

Data-Free Quantization Through Weight Equalization and Bias Correction [Markus *et al.*, ICCV 2019]
Quantizing Deep Convolutional Networks for Efficient Inference: A Whitepaper [Raghuraman Krishnamoorthi, arXiv 2018]
8-bit Inference with TensorRT [Szymon Migacz, 2017]

Post-Training INT8 Linear Quantization

Activation		Symmetric	Asymmertric
		Per-Tensor	Per-Tensor
		Minimize KL-Divergence	Exponential Moving Average (EMA)
Weight		Symmetric	Symmetric
		Per-Tensor	Per-Channel
Neural Network			
	MobileNetV1	-	-11.8%
	MobileNetV2	-	-2.1%

Smaller models seem to not respond as well to post-training quantization, presumabley due to their smaller representational capacity.

How should we improve performance of quantized models?

Data-Free Quantization Through Weight Equalization and Bias Correction [Markus *et al.*, ICCV 2019]
Quantizing Deep Convolutional Networks for Efficient Inference: A Whitepaper [Raghuraman Krishnamoorthi, arXiv 2018]
8-bit Inference with TensorRT [Szymon Migacz, 2017]

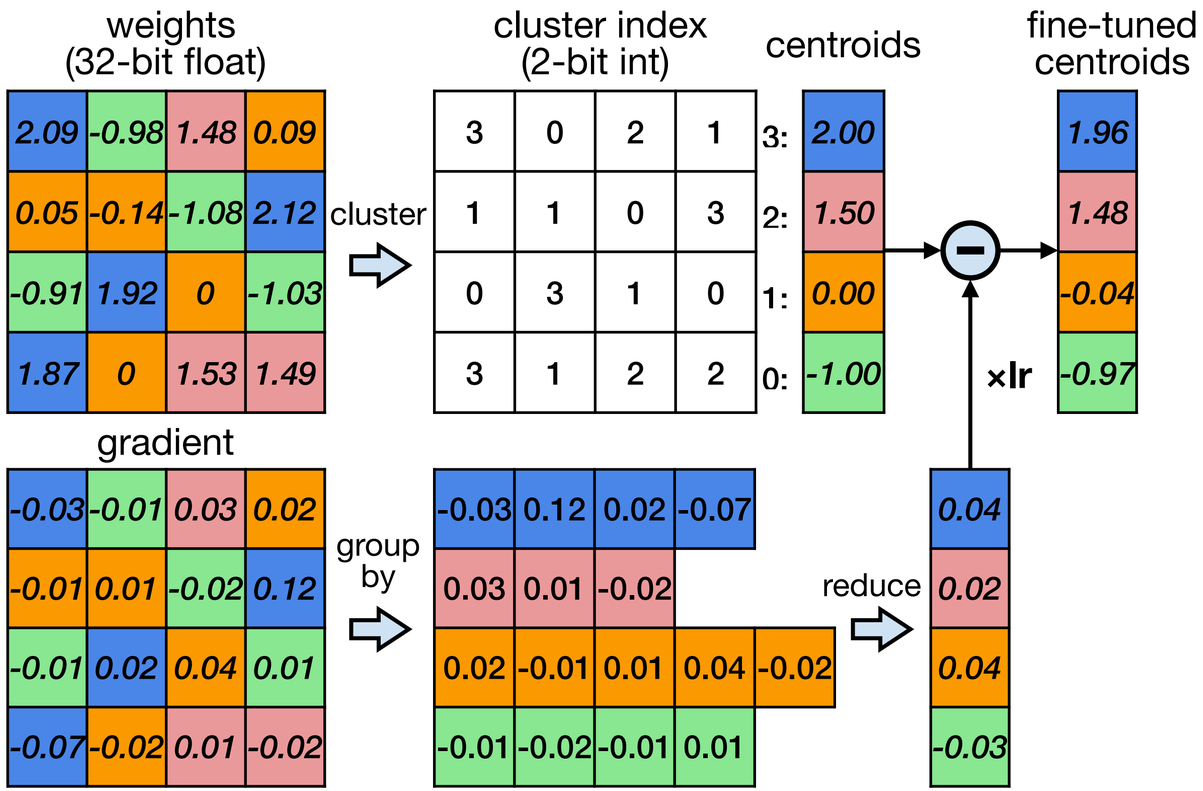
Quantization-Aware Training

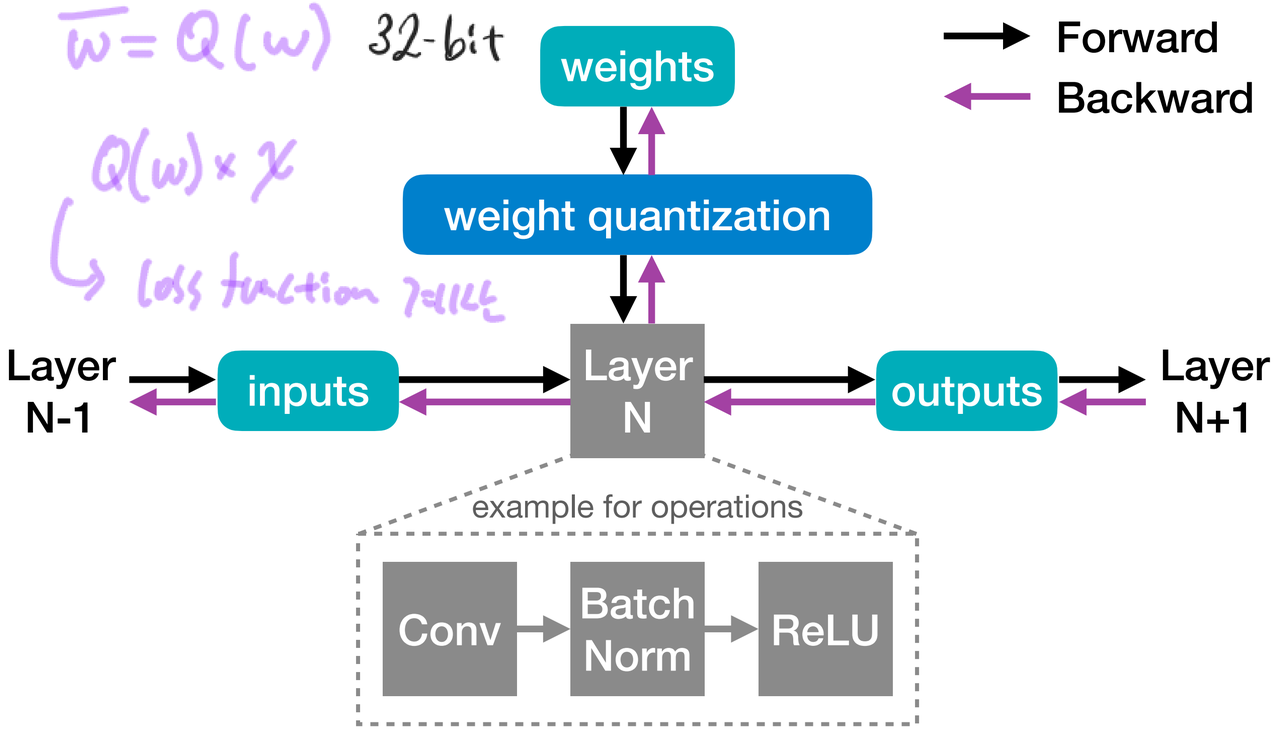
How should we improve performance of quantized models?

Quantization-Aware Training

Train the model taking quantization into consideration

- To minimize the loss of accuracy, especially aggressive quantization with 4 bits and lower bit width, neural network will be trained/fine-tuned with quantized weights and activations.
- Usually, fine-tuning a pre-trained floating point model provides better accuracy than training from scratch.



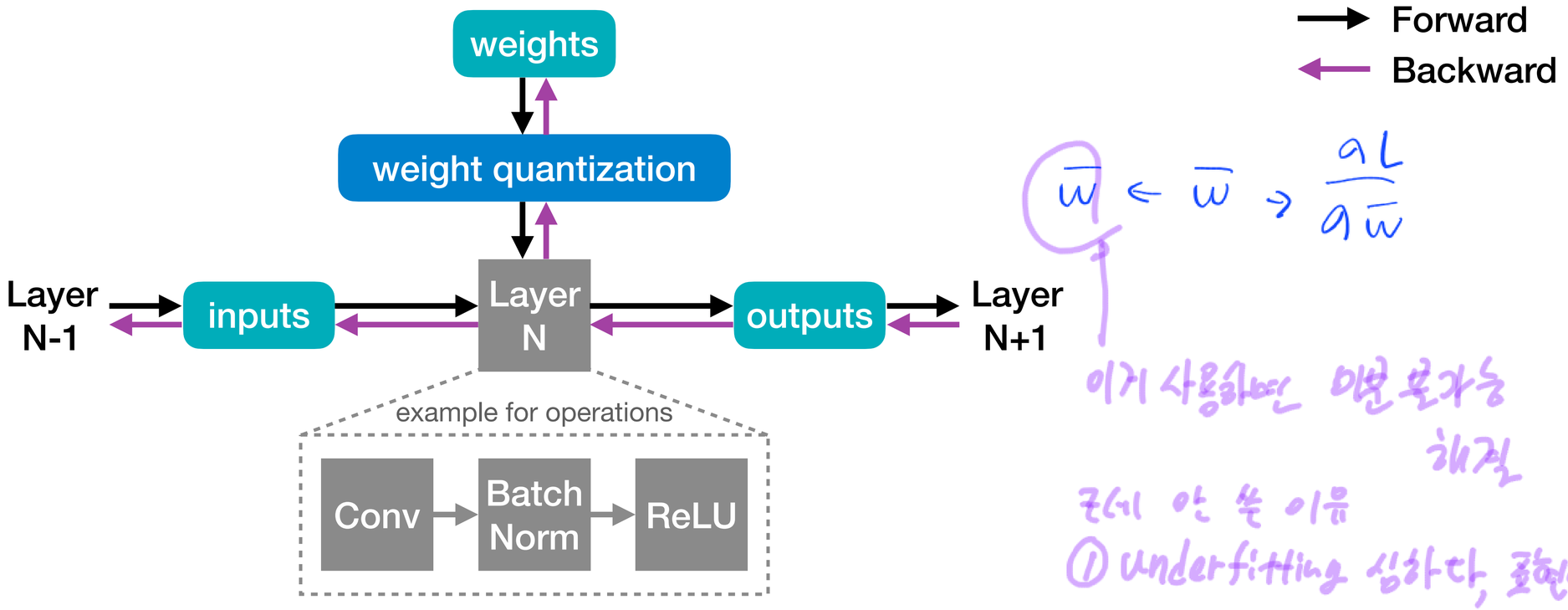


Deep Compression [Han *et al.*, ICLR 2016]

Quantization-Aware Training

Train the model taking quantization into consideration

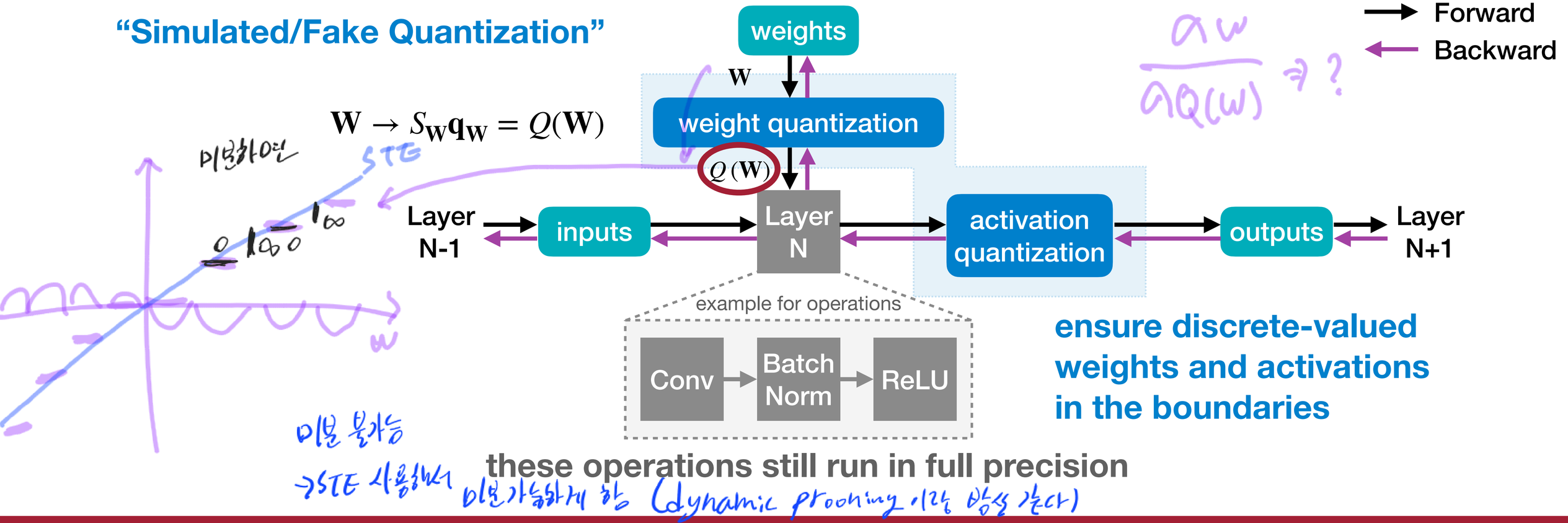
- A full precision copy of the weights W is maintained throughout the training.
- The small gradients are accumulated without loss of precision.
- Once the model is trained, only the quantized weights are used for inference.



Quantization-Aware Training

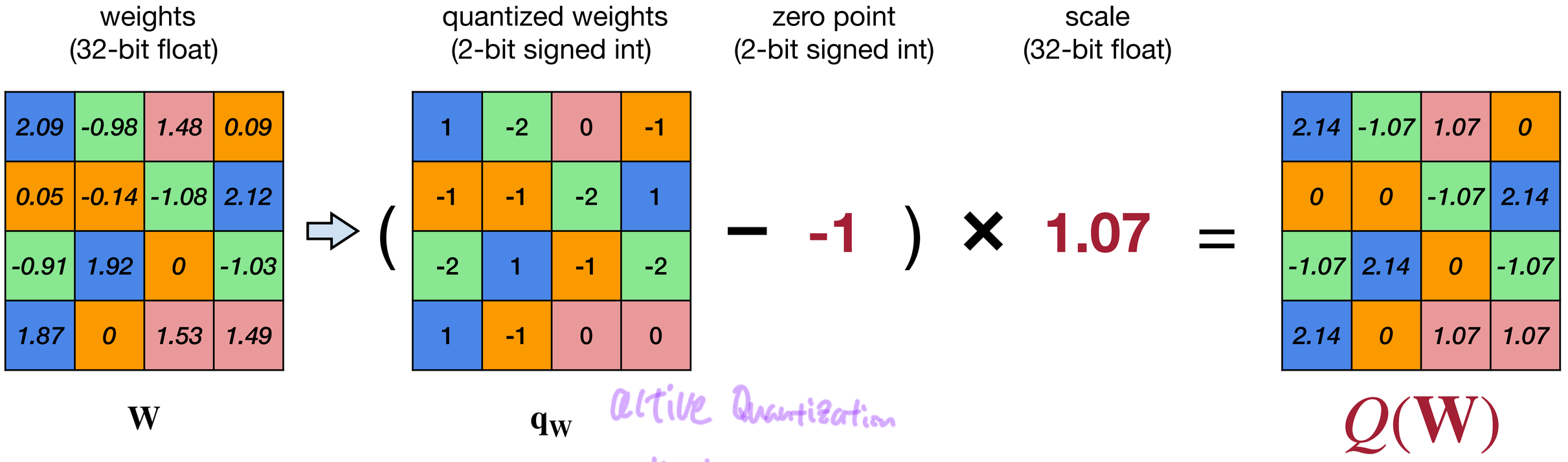
Train the model taking quantization into consideration

- A full precision copy of the weights W is maintained throughout the training.
- The small gradients are accumulated without loss of precision.
- Once the model is trained, only the quantized weights are used for inference.



Linear Quantization

An affine mapping of integers to real numbers $r = S(q - Z)$



Active Quantization

: 입력마다 step size 바꾸는 것

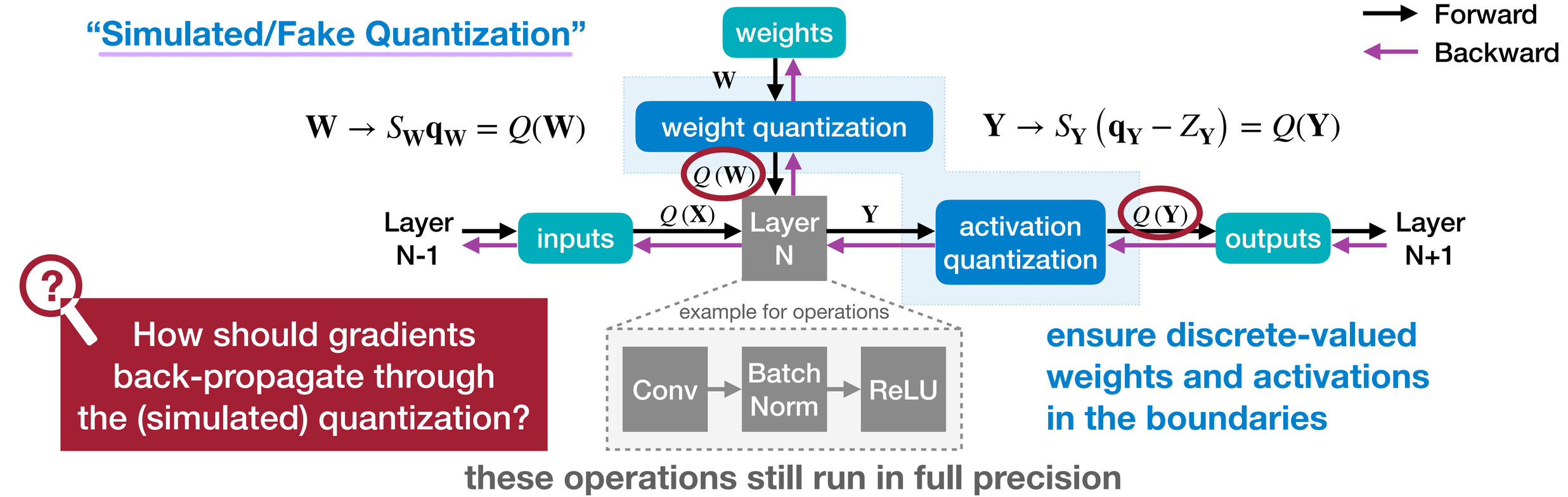
계산량 (edge 환경)

미리 정해둔 값으로 (static) => Cost↓

Quantization-Aware Training

Train the model taking quantization into consideration

- A full precision copy of the weights W is maintained throughout the training.
- The small gradients are accumulated without loss of precision.
- Once the model is trained, only the quantized weights are used for inference.



Straight-Through Estimator (STE)

- Quantization is discrete-valued, and thus the derivative is 0 almost everywhere.

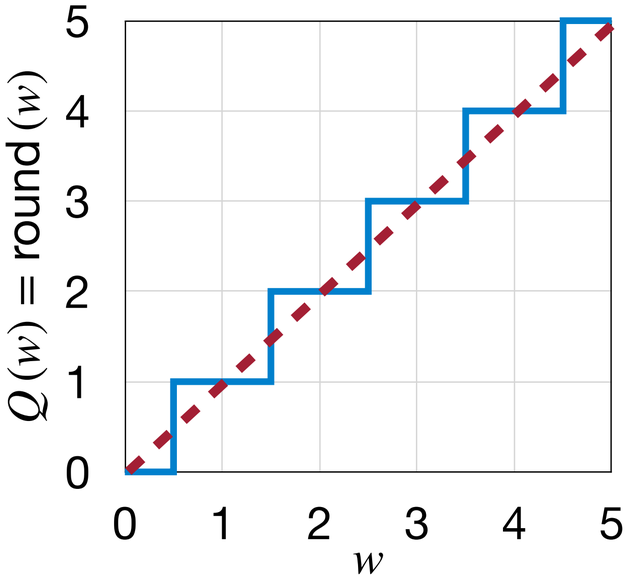
$$\frac{\partial Q(W)}{\partial W} = 0$$

  - The neural network will learn nothing since gradients become 0 and the weights won't get updated.

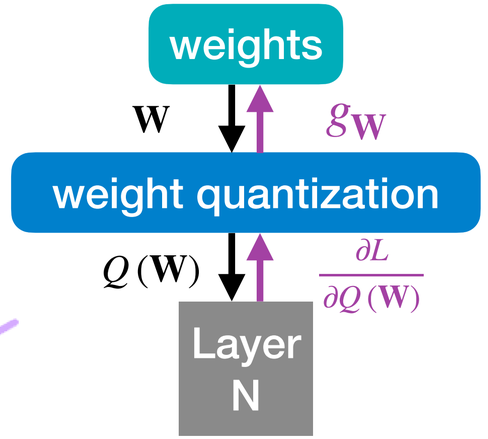
$$g_{\mathbf{W}} = \frac{\partial L}{\partial \mathbf{W}} = \frac{\partial L}{\partial Q(\mathbf{W})} \cdot \frac{\partial Q(\mathbf{W})}{\partial \mathbf{W}} = 0$$

- Straight-Through Estimator (STE) simply passes the gradients through the quantization as if it had been the *identity* function.

$$g_{\mathbf{W}} = \frac{\partial L}{\partial \mathbf{W}} = \frac{\partial L}{\partial Q(\mathbf{W})}$$



$\frac{\partial L}{\partial Q(w)} = \frac{\partial L}{\partial w}$ 동일

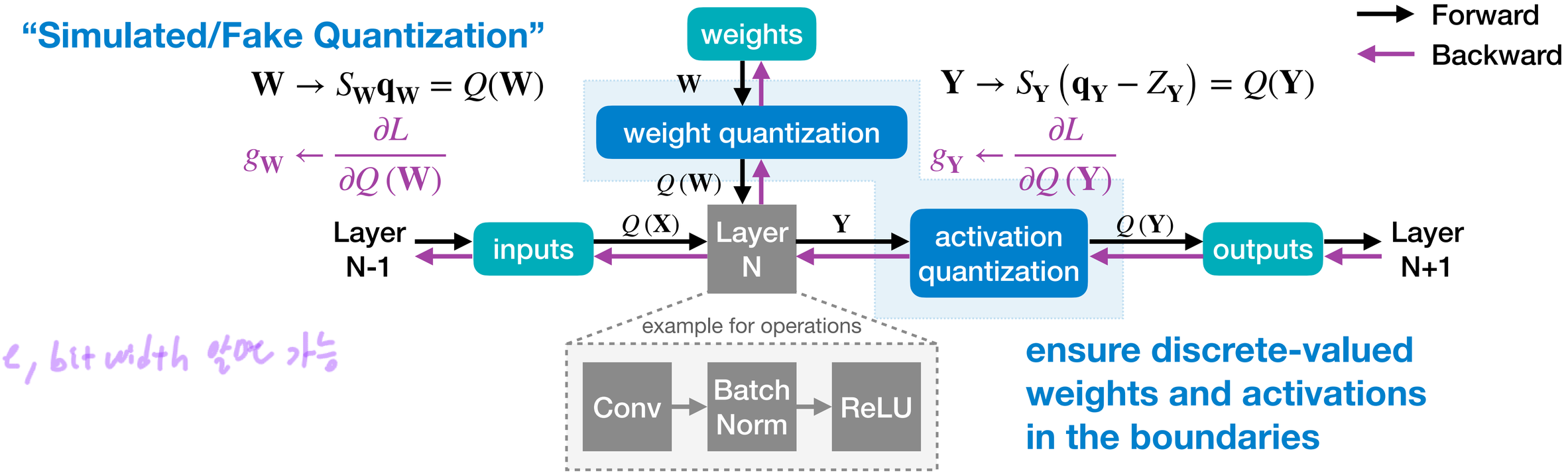


Neural Networks for Machine Learning [Hinton *et al.*, Coursera Video Lecture, 2012]
Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation [Bengio, arXiv 2013]

Quantization-Aware Training

Train the model taking quantization into consideration

- A full precision copy of the weights is maintained throughout the training.
- The small gradients are accumulated without loss of precision.
- Once the model is trained, only the quantized weights are used for inference.

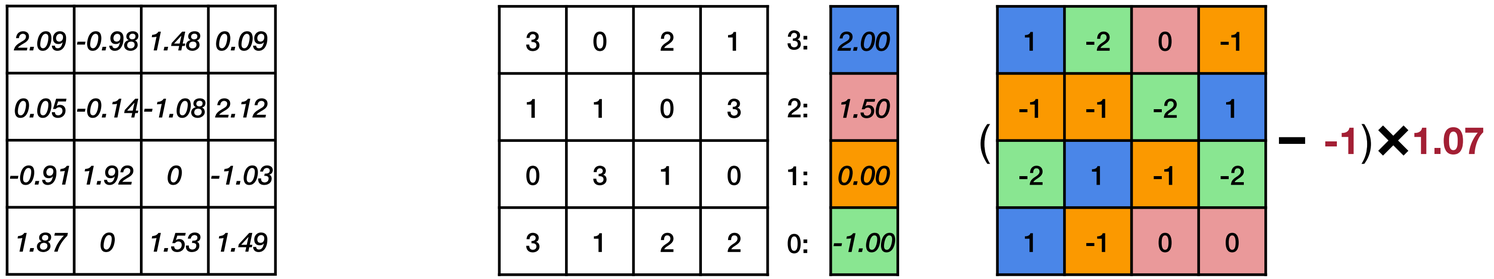


INT8 Linear Quantization-Aware Training

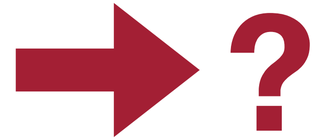
Neural Network	Floating-Point	Post-Training Quantization		Quantization-Aware Training	
		Asymmetric	Symmetric	Asymmetric	Symmetric
		Per-Tensor	Per-Channel	Per-Tensor	Per-Channel
MobileNetV1	70.9%	0.1%	59.1%	70.0%	70.7%
MobileNetV2	71.9%	0.1%	69.8%	70.9%	71.1%
NASNet-Mobile	74.9%	72.2%	72.1%	73.0%	73.0%

Quantizing Deep Convolutional Networks for Efficient Inference: A Whitepaper [Raghuraman Krishnamoorthi, arXiv 2018]

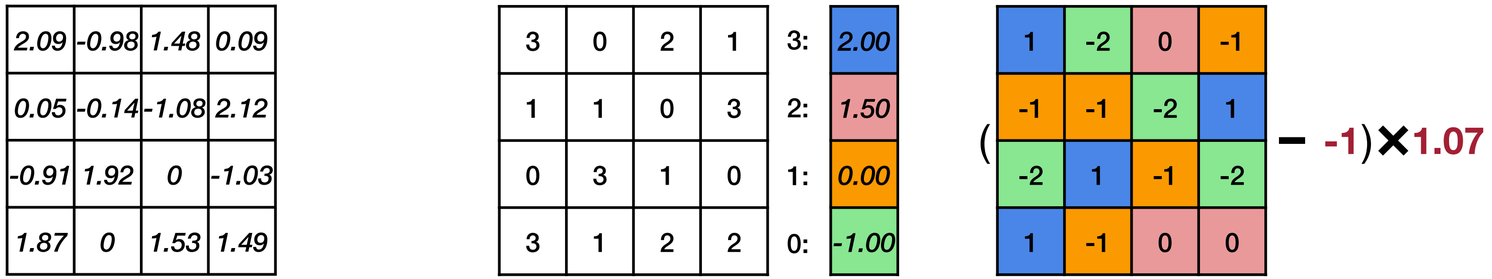
Neural Network Quantization



		K-Means-based Quantization	**Linear Quantization**
Storage	Floating-Point Weights	Integer Weights; Floating-Point Codebook	Integer Weights
Computation	Floating-Point Arithmetic	Floating-Point Arithmetic	Integer Arithmetic



Neural Network Quantization



		K-Means-based Quantization	Linear Quantization	Binary/Ternary Quantization
Storage	Floating-Point Weights	Integer Weights; Floating-Point Codebook	Integer Weights	Binary/Ternary Weights
Computation	Floating-Point Arithmetic	Floating-Point Arithmetic	Integer Arithmetic	Bit Operations

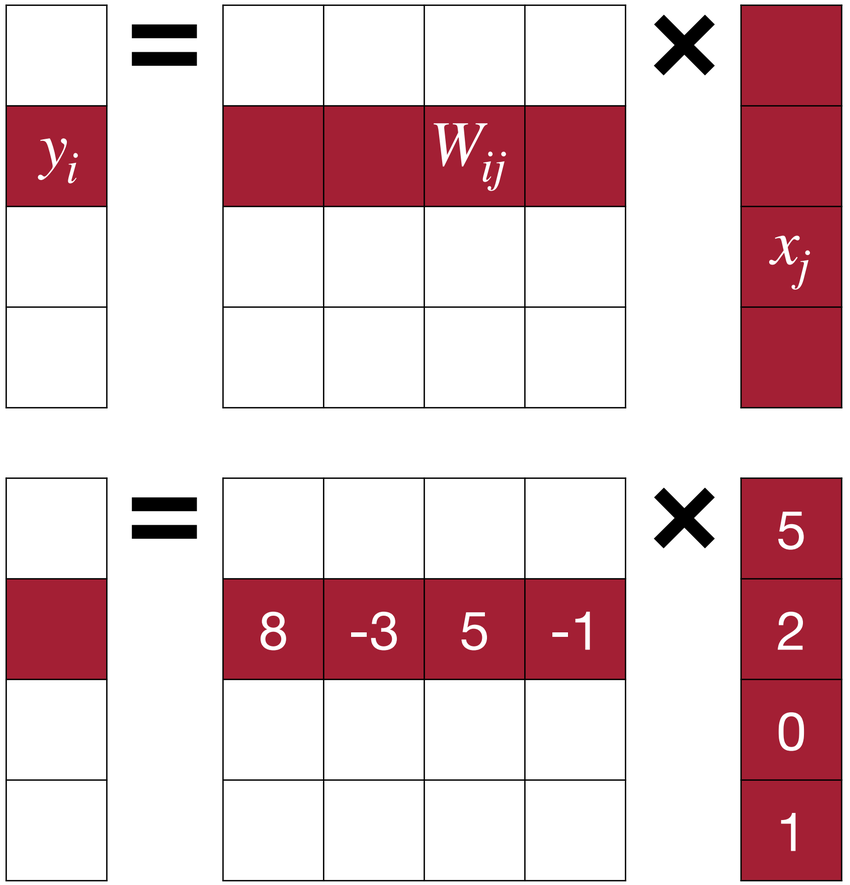
Binary/Ternary Quantization

Can we push the quantization precision to 1 bit?

Can quantization bit width go even lower?

$$y_i = \sum_j W_{ij} \cdot x_j$$

$$= 8\times5 + (-3)\times2 + 5\times0 + (-1)\times1$$

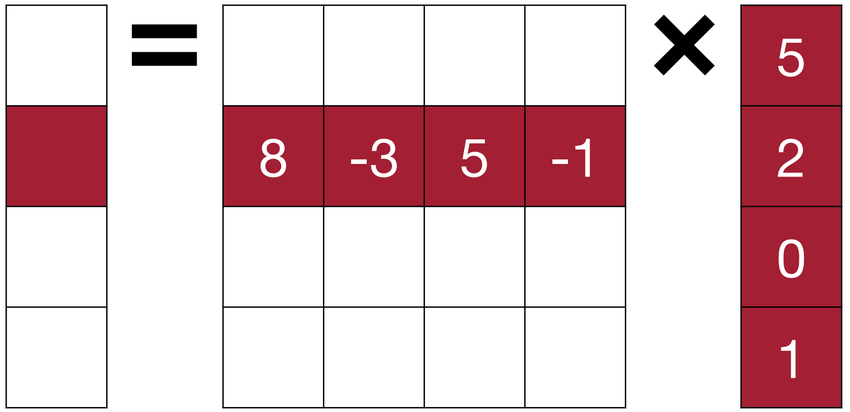


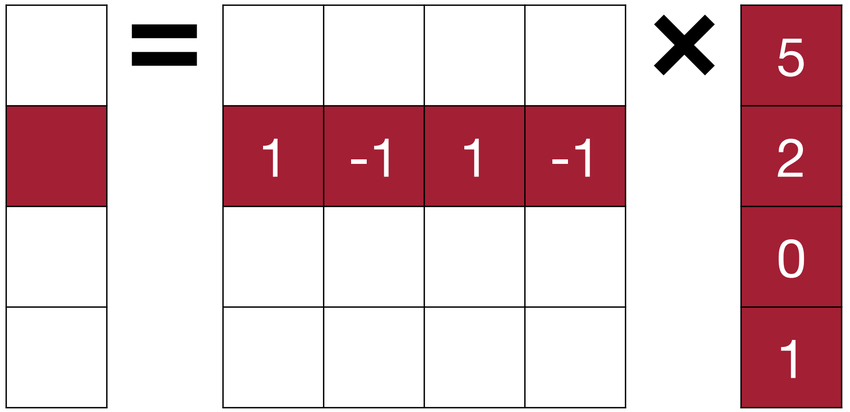
input	weight	operations	memory	computation
$\mathbb{R}$	$\mathbb{R}$	+ ×	1×	1×

If weights are quantized to +1 and -1

$$y_i = \sum_j W_{ij} \cdot x_j$$

$$= 5 - 2 + 0 - 1$$





input	weight	operations	memory	computation
$\mathbb{R}$	$\mathbb{R}$	+ ×	1×	1×
$\mathbb{R}$	$\mathbb{B}$	+ -	~32× less	~2× less

BinaryConnect: Training Deep Neural Networks with Binary Weights during Propagations [Courbariaux *et al.*, NeurIPS 2015]
XNOR-Net: ImageNet Classification using Binary Convolutional Neural Networks [Rastegari *et al.*, ECCV 2016]

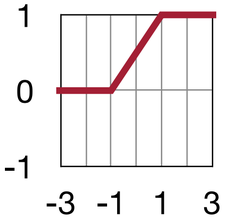
Binarization

- **Deterministic Binarization**
  - directly computes the bit value based on a threshold, usually 0, resulting in a sign function.

$$q = \text{sign}\,(r) = \begin{cases} +1, & r \geq 0 \\ -1, & r < 0 \end{cases}$$

- **Stochastic Binarization**
  - use global statistics or the value of input data to determine the probability of being -1 or +1
    - *e.g.*, in Binary Connect (BC), probability is determined by hard sigmoid function $\sigma(r)$

$$q = \begin{cases} +1, & \text{with probability } p = \sigma(r) \\ -1, & \text{with probability } 1 - p \end{cases}, \quad \text{where } \sigma(r) = \min(\max(\frac{r+1}{2}, 0), 1)$$



  - harder to implement as it requires the hardware to generate random bits when quantizing.

BinaryConnect: Training Deep Neural Networks with Binary Weights during Propagations [Courbariaux *et al.*, NeurIPS 2015]
BinaryNet: Training Deep Neural Networks with Weights and Activations Constrained to +1 or –1. [Courbariaux *et al.*, Arxiv 2016]

Minimizing Quantization Error in Binarization

weights
(32-bit float)

2.09	*-0.98*	*1.48*	*0.09*
0.05	*-0.14*	*-1.08*	*2.12*
-0.91	*1.92*	*0*	*-1.03*
1.87	*0*	*1.53*	*1.49*

$\mathbf{W}$ → $\mathbf{W}^{\mathbb{B}}$

binary weights
(1-bit)

1	-1	1	1
1	-1	-1	1
-1	1	1	-1
1	1	1	1

$$\mathbf{W}^{\mathbb{B}} = \text{sign}(\mathbf{W})$$

$$\alpha = \frac{1}{n}\|\mathbf{W}\|_1$$

$\alpha\mathbf{W}^{\mathbb{B}}$

1	-1	1	1
1	-1	-1	1
-1	1	1	-1
1	1	1	1

$$\|\mathbf{W} - \mathbf{W}^{\mathbb{B}}\|_F^2 = 9.28$$

scale
(32-bit float)

× **1.05** $= \frac{1}{16}\|\mathbf{W}\|_1$

$$\|\mathbf{W} - \alpha\mathbf{W}^{\mathbb{B}}\|_F^2 = 9.24$$

AlexNet-based Network	ImageNet Top-1 Accuracy Delta
BinaryConnect	-21.2%
Binary Weight Network (BWN)	0.2%

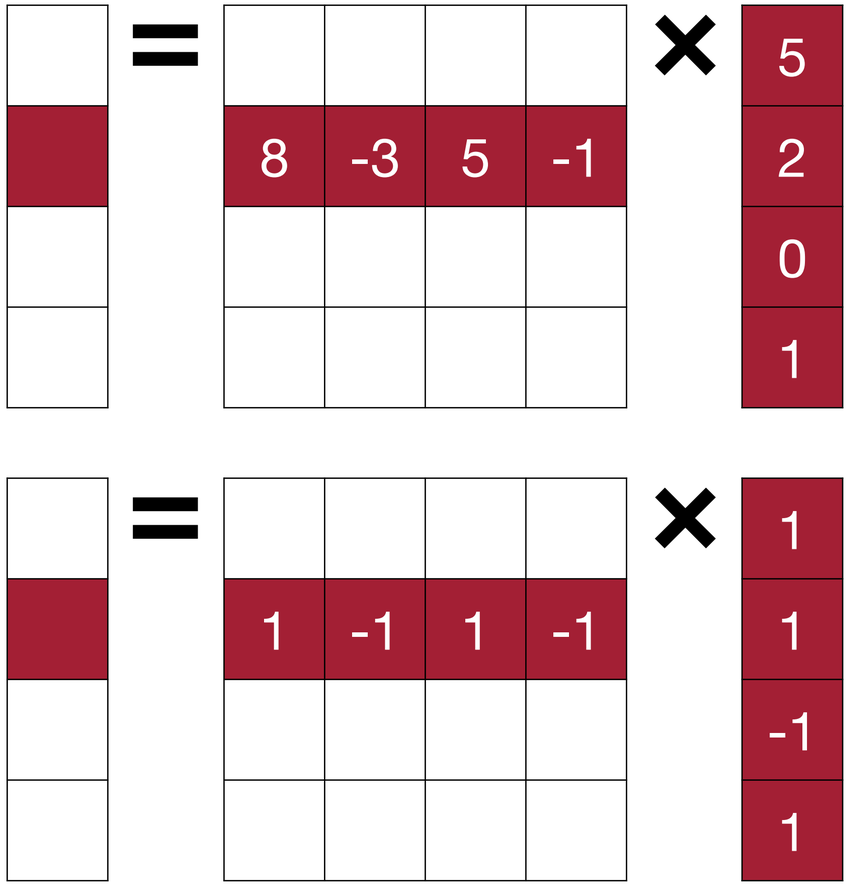
XNOR-Net: ImageNet Classification using Binary Convolutional Neural Networks [Rastegari *et al.*, ECCV 2016]

If both activations and weights are binarized

$$y_i = \sum_j W_{ij} \cdot x_j$$

$$= 1\times1 + (-1)\times1 + 1\times(-1) + (-1)\times1$$

$$= 1 + (-1) + (-1) + (-1) = -2$$



XNOR-Net: ImageNet Classification using Binary Convolutional Neural Networks [Rastegari *et al.*, ECCV 2016]

If both activations and weights are binarized

$$y_i = \sum_j W_{ij} \cdot x_j$$

$$= 1\times1 + (-1)\times1 + 1\times(-1) + (-1)\times1$$

$$= 1 + (-1) + (-1) + (-1) = -2$$

W	X	Y=WX
1	1	1
1	-1	-1
-1	-1	1
-1	1	-1

b_W	b_X	XNOR(b_W, b_X)
1	1	1
1	0	0
0	0	1
0	1	0

XNOR-Net: ImageNet Classification using Binary Convolutional Neural Networks [Rastegari *et al.*, ECCV 2016]

If both activations and weights are binarized

$$y_i = \sum_j W_{ij} \cdot x_j$$

= 1×1 + (-1)×1 + 1×(-1) + (-1)×1 = -2

= 1 xnor 1 + 0 xnor 1 + 1 xnor 0 + 0 xnor 1

= 1 + 0 + 0 + 0 = 1

?

W	X	Y=WX
1	1	1
1	-1	-1
-1	-1	1
-1	1	-1

b_W	b_X	XNOR(b_W, b_X)
1	1	1
1	0	0
0	0	1
0	1	0

XNOR-Net: ImageNet Classification using Binary Convolutional Neural Networks [Rastegari *et al.*, ECCV 2016]

If both activations and weights are binarized

$$y_i = \sum_j W_{ij} \cdot x_j$$

$$= 1\times1 + (-1)\times1 + 1\times(-1) + (-1)\times1$$

$$= 1 + (-1) + (-1) + (-1) = -2$$

$$y_i = -n + 2 \cdot \sum_j W_{ij} \text{ xnor } x_j$$

$$= 1 \text{ xnor } 1 + 0 \text{ xnor } 1 + 1 \text{ xnor } 0 + 0 \text{ xnor } 1$$

$$= 1 + 0 + 0 + 0 = 1 \times 2 + (+2) = -4 = -2$$

Assuming -1 -1 -1 -1 →

W	X	Y=WX
1	1	1
1	-1	-1
-1	-1	1
-1	1	-1

b_W	b_X	XNOR(b_W, b_X)
1	1	1
1	0	0
0	0	1
0	1	0

XNOR-Net: ImageNet Classification using Binary Convolutional Neural Networks [Rastegari *et al.*, ECCV 2016]

If both activations and weights are binarized

$$y_i = -n + 2 \cdot \sum_j W_{ij} \text{ xnor } x_j \quad \rightarrow \quad y_i = -n + \text{popcount}\left(W_i \text{ xnor } x\right) \ll 1$$

= -4 + 2 × (1 xnor 1 + 0 xnor 1 + 1 xnor 0 + 0 xnor 1)

= -4 + 2 × (1 + 0 + 0 + 0) = -2

→ popcount: return the number of 1

W	X	Y=WX
1	1	1
1	-1	-1
-1	-1	1
-1	1	-1

b_W	b_X	XNOR(b_W, b_X)
1	1	1
1	0	0
0	0	1
0	1	0

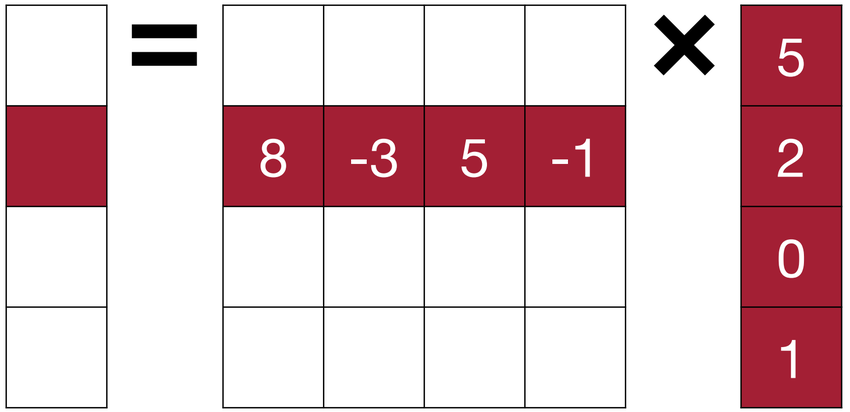
XNOR-Net: ImageNet Classification using Binary Convolutional Neural Networks [Rastegari *et al.*, ECCV 2016]

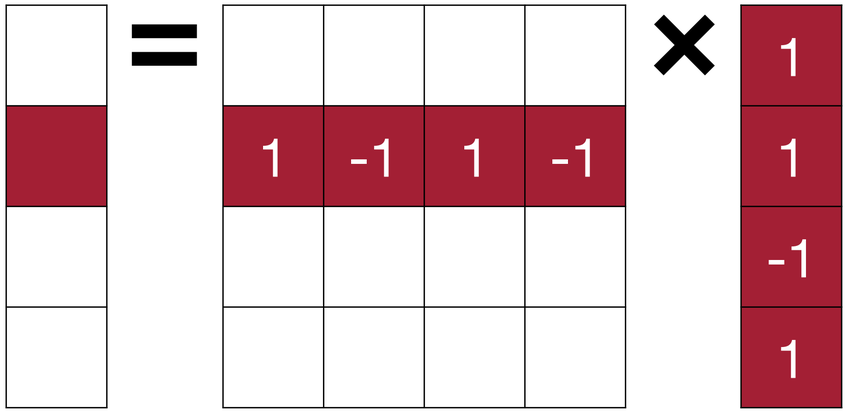
If both activations and weights are binarized

$$y_i = -n + \text{popcount}\left(W_i \text{ xnor } x\right) \ll 1$$

= -4 + popcount(1010 xnor 1101) ≪ 1

= -4 + popcount(1000) ≪ 1 = -4 + 2 = -2





input	weight	operations	memory	computation
$\mathbb{R}$	$\mathbb{R}$	+ ×	1×	1×
$\mathbb{R}$	$\mathbb{B}$	+ -	~32× less	~2× less
$\mathbb{B}$	$\mathbb{B}$	xnor, popcount	~32× less	~58× less

XNOR-Net: ImageNet Classification using Binary Convolutional Neural Networks [Rastegari *et al.*, ECCV 2016]

Accuracy Degradation of Binarization

Neural Network	Quantization	Bit-Width		ImageNet Top-1 Accuracy Delta
		W	A	
AlexNet	BWN	1	32	0.2%
	BNN	1	1	-28.7%
	XNOR-Net	1	1	-12.4%
GoogleNet	BWN	1	32	-5.80%
	BNN	1	1	-24.20%
ResNet-18	BWN	1	32	-8.5%
	XNOR-Net	1	1	-18.1%

* BWN: Binary Weight Network with scale for weight binarization
* BNN: Binarized Neural Network without scale factors
* XNOR-Net: scale factors for both activation and weight binarization

Binarized Neural Networks: Training Deep Neural Networks with Weights and Activations Constrained to +1 or –1. [Courbariaux *et al.*, Arxiv 2016]
XNOR-Net: ImageNet Classification using Binary Convolutional Neural Networks [Rastegari *et al.*, ECCV 2016]

Ternary Weight Networks (TWN)

Weights are quantized to +1, -1 and 0

$$q = \begin{cases} r_t, & r > \Delta \\ 0, & |r| \le \Delta, \\ -r_t, & r < -\Delta \end{cases} \quad \text{where } \Delta = 0.7 \times \mathbb{E}\left(|r|\right), r_t = \mathbb{E}_{|r|>\Delta}\left(|r|\right)$$

weights $\mathbf{W}$ (32-bit float)

2.09	*-0.98*	*1.48*	*0.09*
0.05	*-0.14*	*-1.08*	*2.12*
-0.91	*1.92*	*0*	*-1.03*
1.87	*0*	*1.53*	*1.49*



ternary weights $\mathbf{W}^{\mathbb{T}}$ (2-bit)

1	*-1*	*1*	*0*
0	*0*	*-1*	*1*
-1	*1*	*0*	*-1*
1	*0*	*1*	*1*

$$\Delta = 0.7 \times \frac{1}{16}\|\mathbf{W}\|_1 = 0.73$$

$$\times \quad \mathbf{1.5} = \frac{1}{11}\|\mathbf{W}_{\mathbf{W}^{\mathbb{T}} \neq 0}\|_1$$

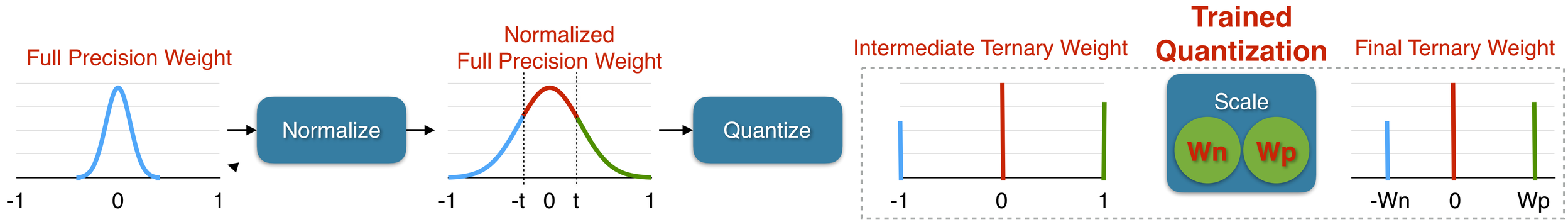
ImageNet Top-1 Accuracy	Full Precision	1 bit (BWN)	2 bit (TWN)
ResNet-18	69.6	60.8	65.3

Ternary Weight Networks [Li *et al.*, Arxiv 2016]

Trained Ternary Quantization (TTQ)

- Instead of using fixed scale r_t, TTQ introduces two *trainable* parameters w_p and w_n to represent the positive and negative scales in the quantization.

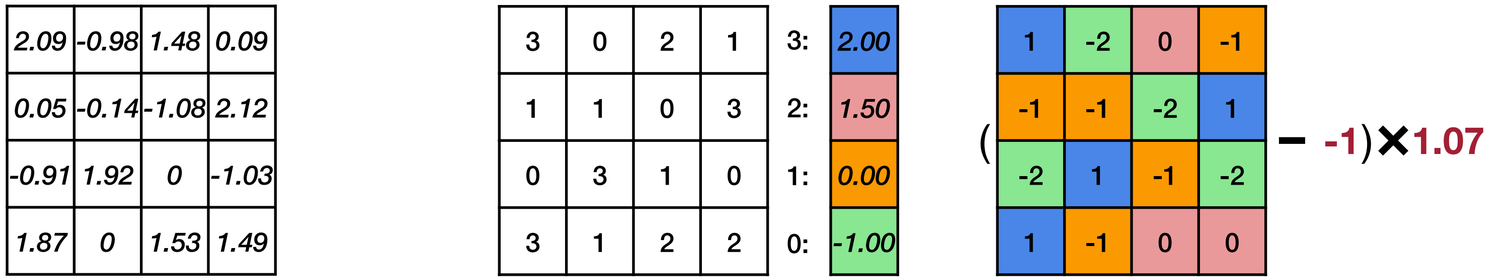
$$q = \begin{cases} w_p, & r > \Delta \\ 0, & |r| \le \Delta \\ -w_n, & r < -\Delta \end{cases}$$



ImageNet Top-1 Accuracy	Full Precision	1 bit (BWN)	2 bit (TWN)	TTQ
ResNet-18	69.6	60.8	65.3	66.6

Trained Ternary Quantization [Zhu *et al.*, ICLR 2017]

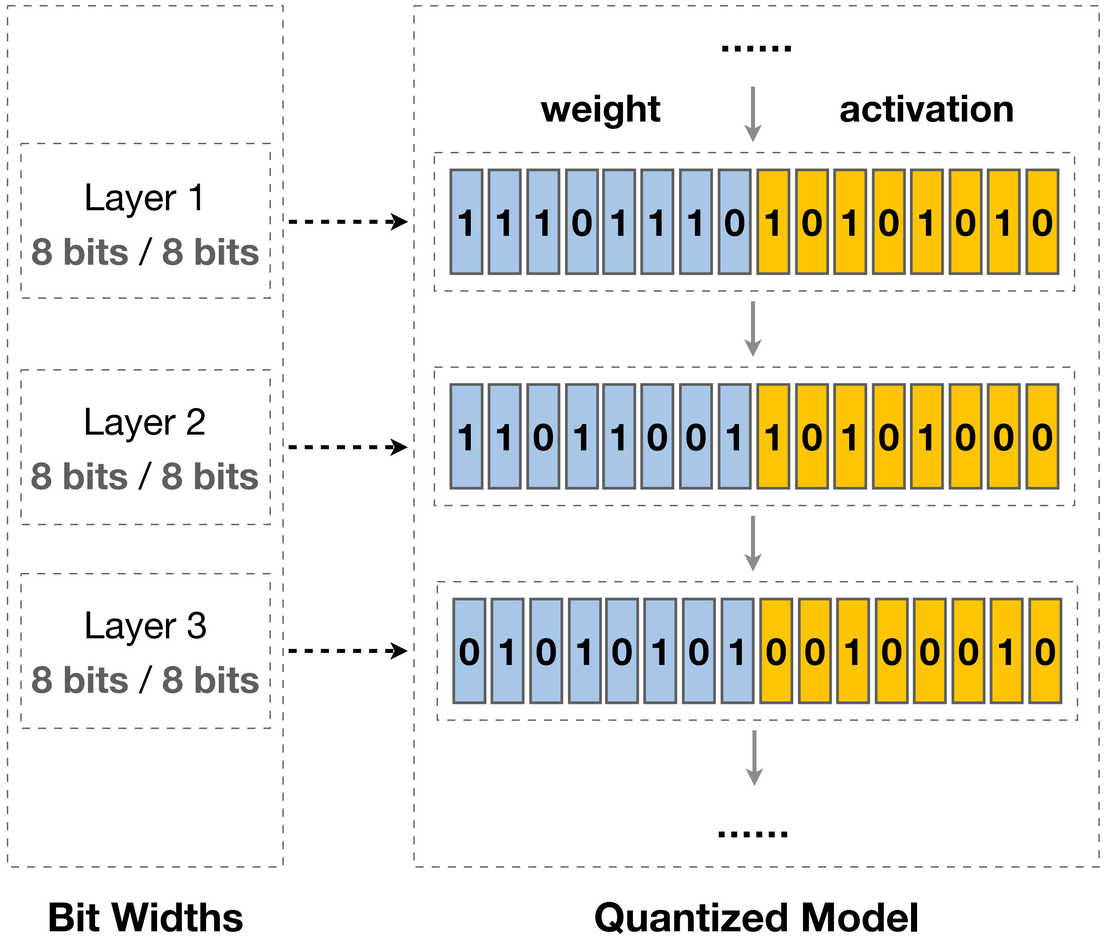
Neural Network Quantization



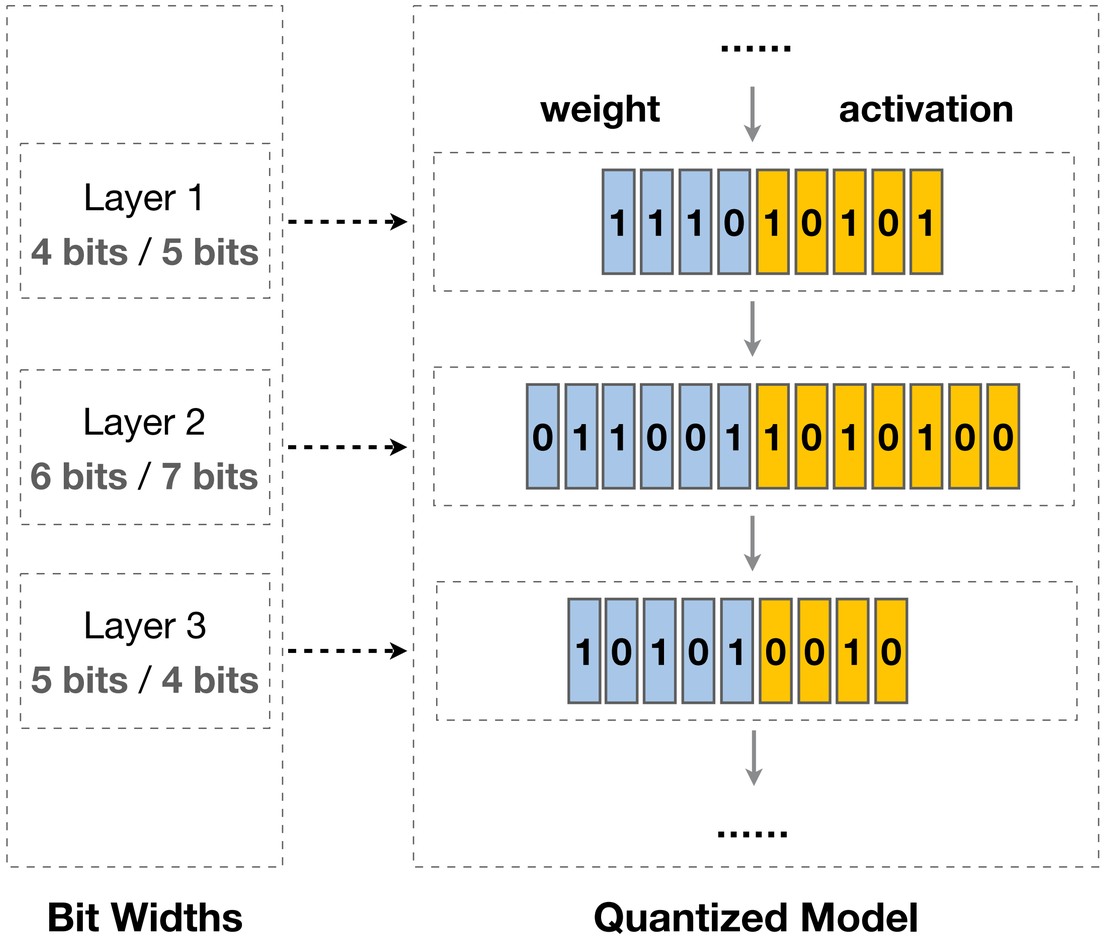
		K-Means-based Quantization	Linear Quantization	Binary/Ternary Quantization
Storage	Floating-Point Weights	Integer Weights; Floating-Point Codebook	Integer Weights	Binary/Ternary Weights
Computation	Floating-Point Arithmetic	Floating-Point Arithmetic	Integer Arithmetic	Bit Operations

Mixed-Precision Quantization

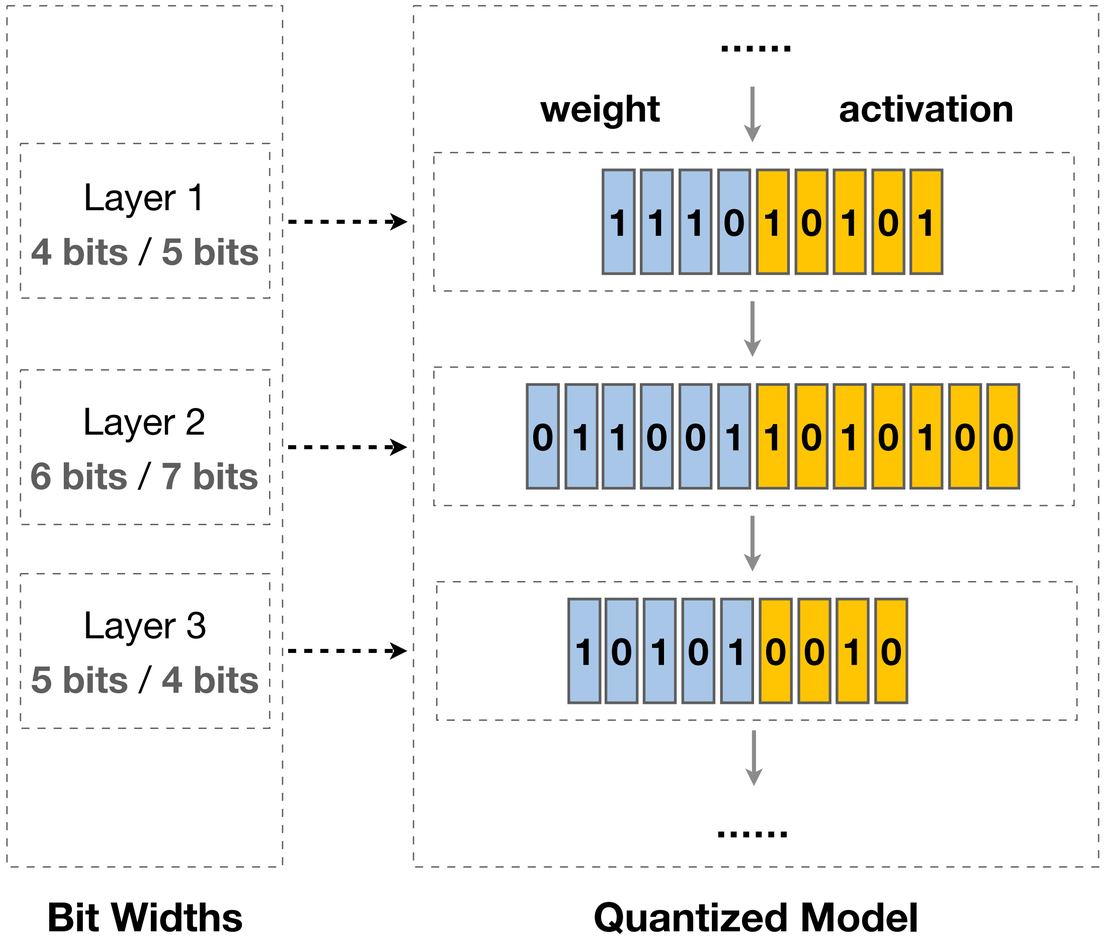
Uniform Quantization



Mixed-Precision Quantization



Challenge: Huge Design Space



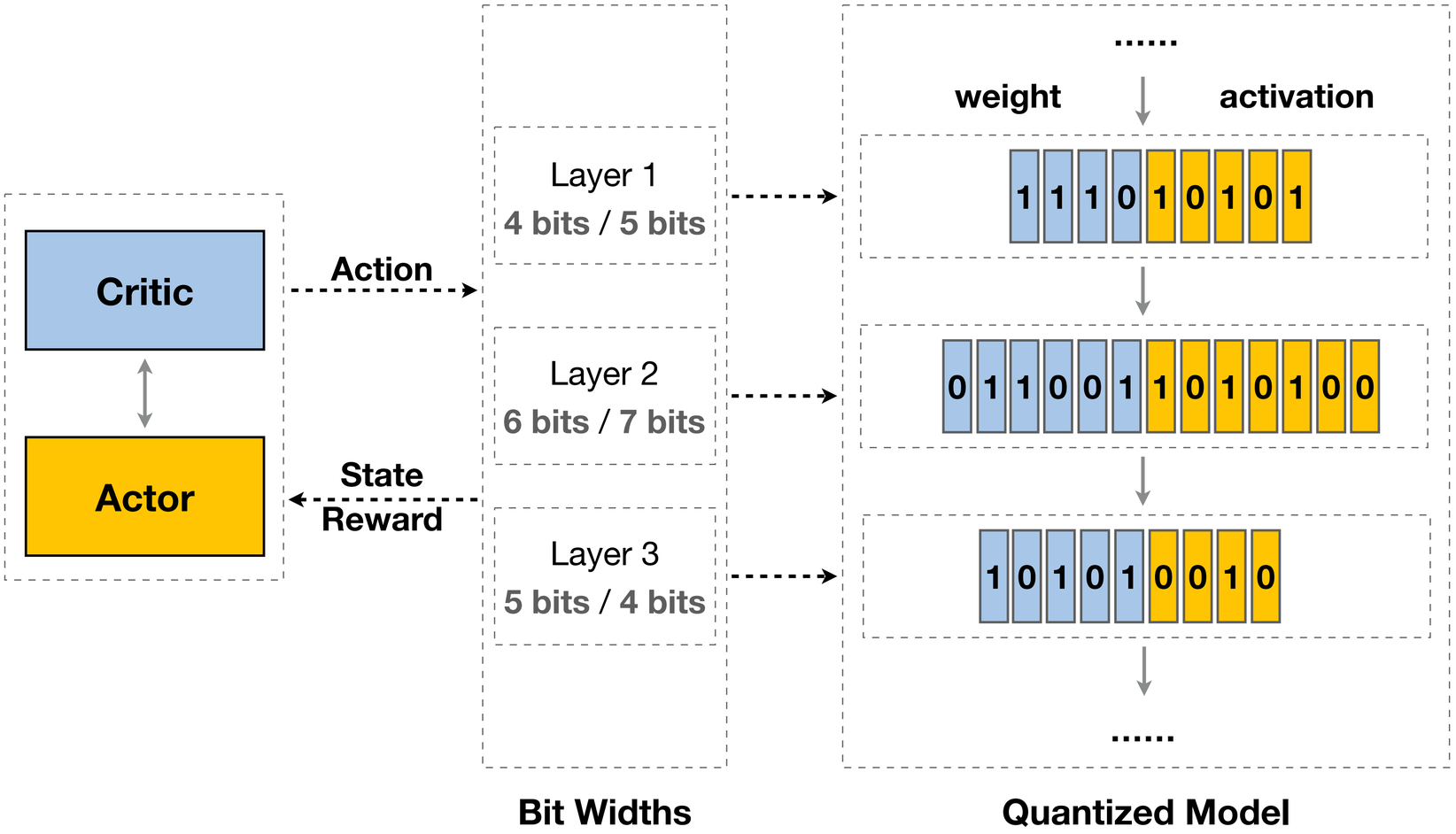
Choices: **8 x 8 = 64**

Choices: **8 x 8 = 64**

Choices: **8 x 8 = 64**

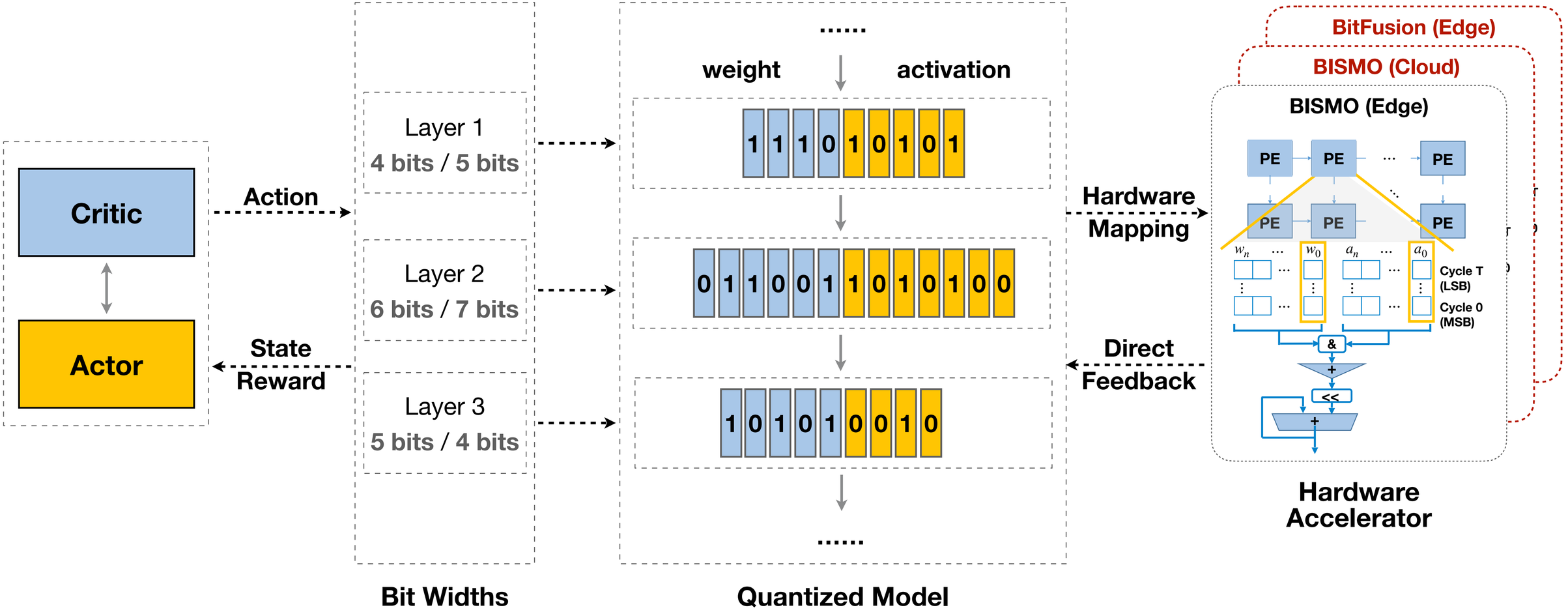
Design Space: 64^n

Solution: Design Automation



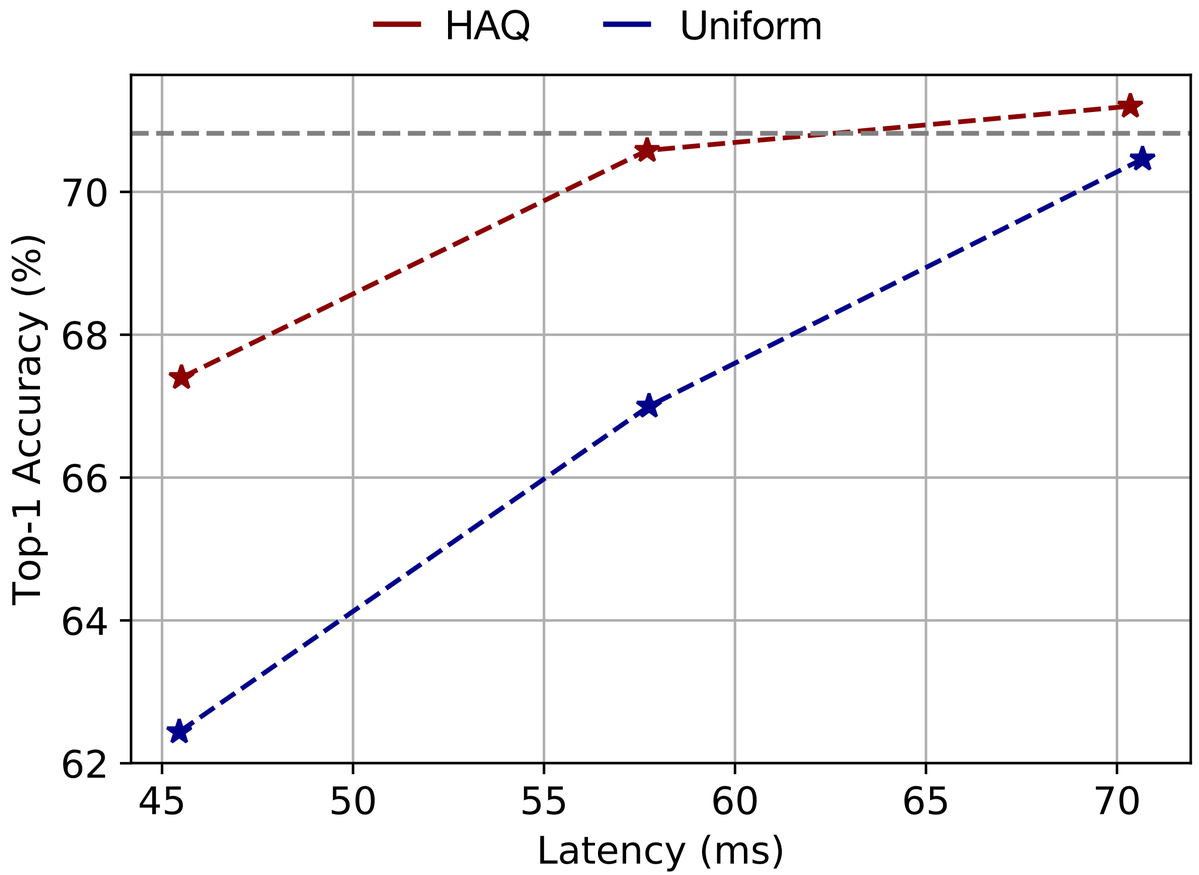
HAQ: Hardware-Aware Automated Quantization with Mixed Precision [Wang *et al.*, CVPR 2019]

Solution: Design Automation



HAQ: Hardware-Aware Automated Quantization with Mixed Precision [Wang *et al.*, CVPR 2019]

HAQ Outperforms Uniform Quantization



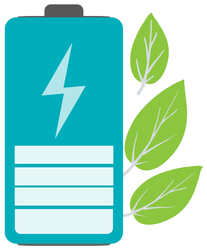
Mixed-Precision Quantized MobileNetV1

HAQ: Hardware-Aware Automated Quantization with Mixed Precision [Wang *et al.*, CVPR 2019]

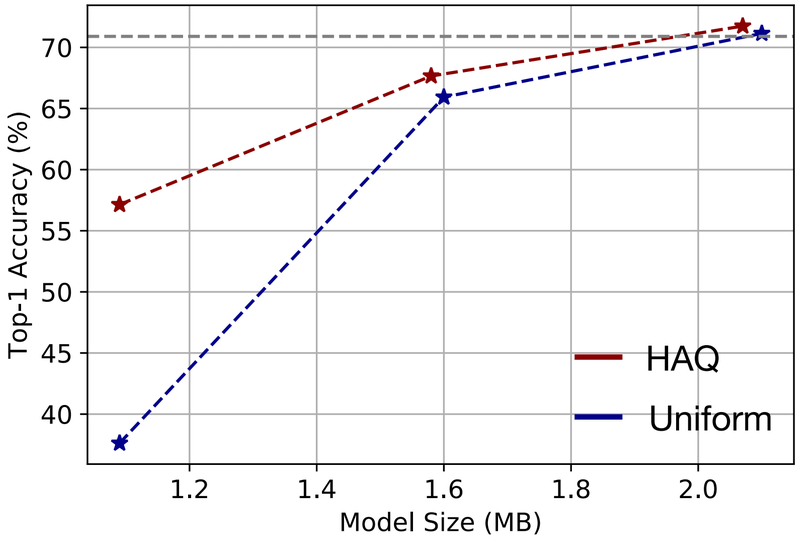
HAQ Supports Multiple Objectives



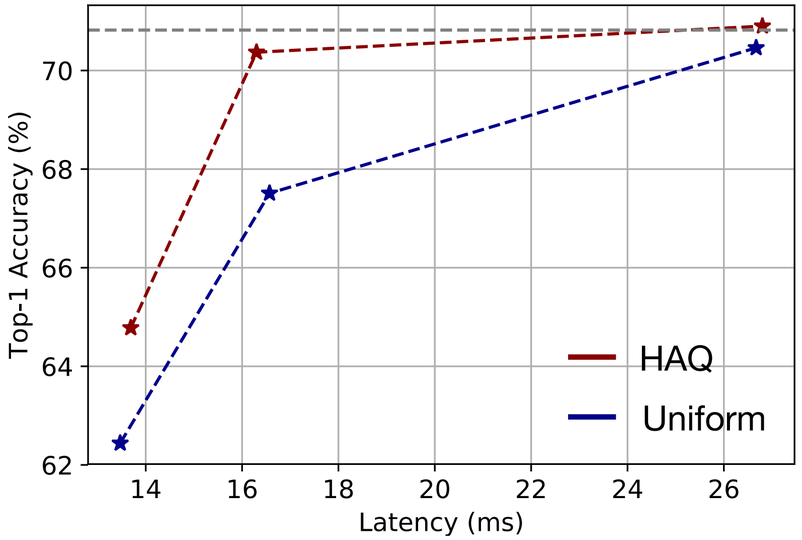




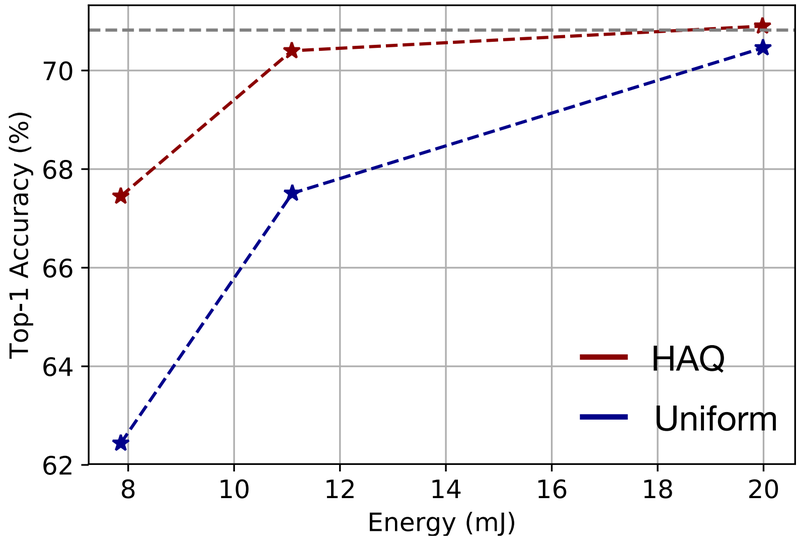
Model Size Constrained



Latency Constrained



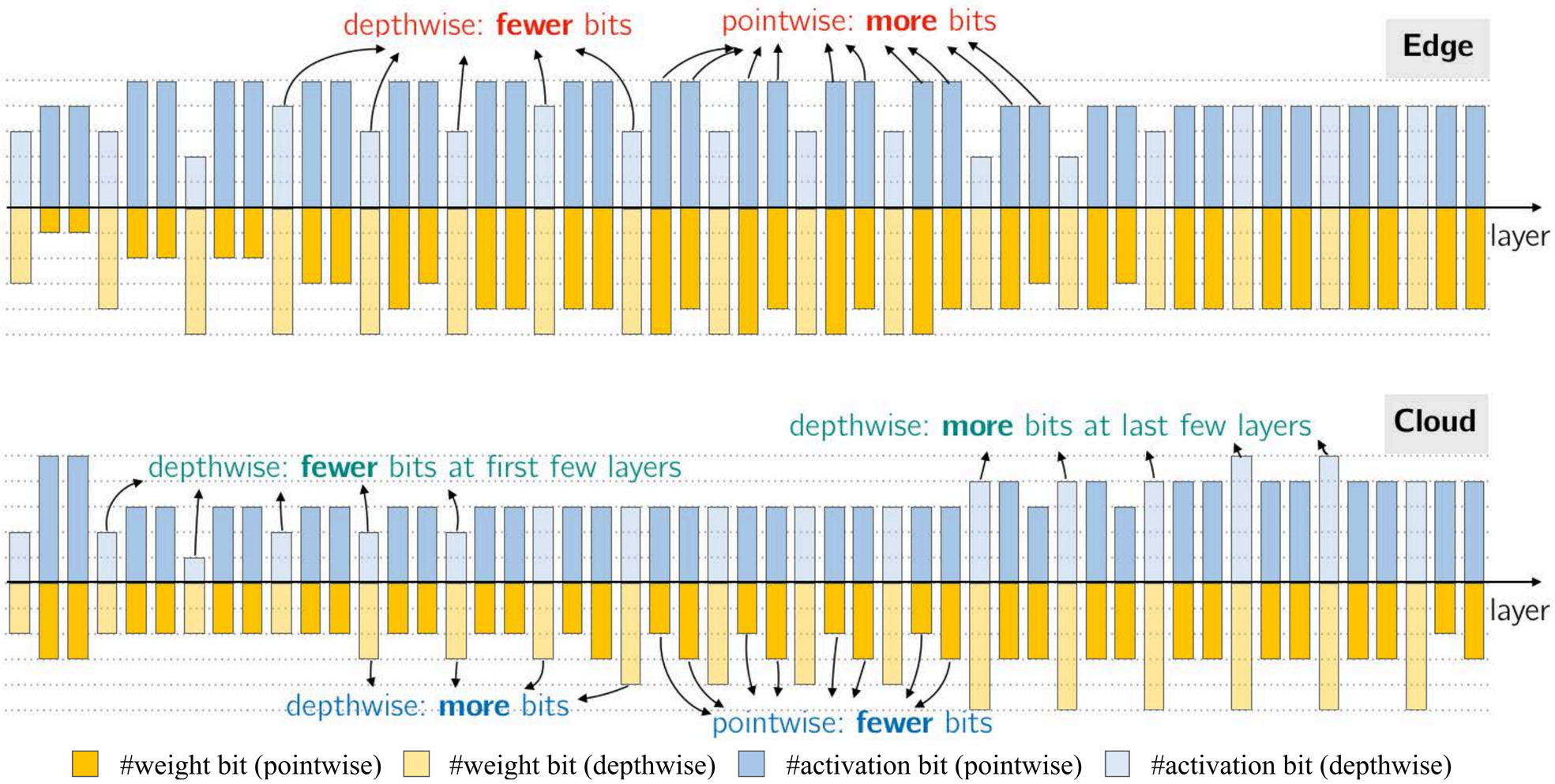
Energy Constrained



Mixed-Precision Quantized MobileNetV1

HAQ: Hardware-Aware Automated Quantization with Mixed Precision [Wang *et al.*, CVPR 2019]

Quantization Policy for Edge and Cloud



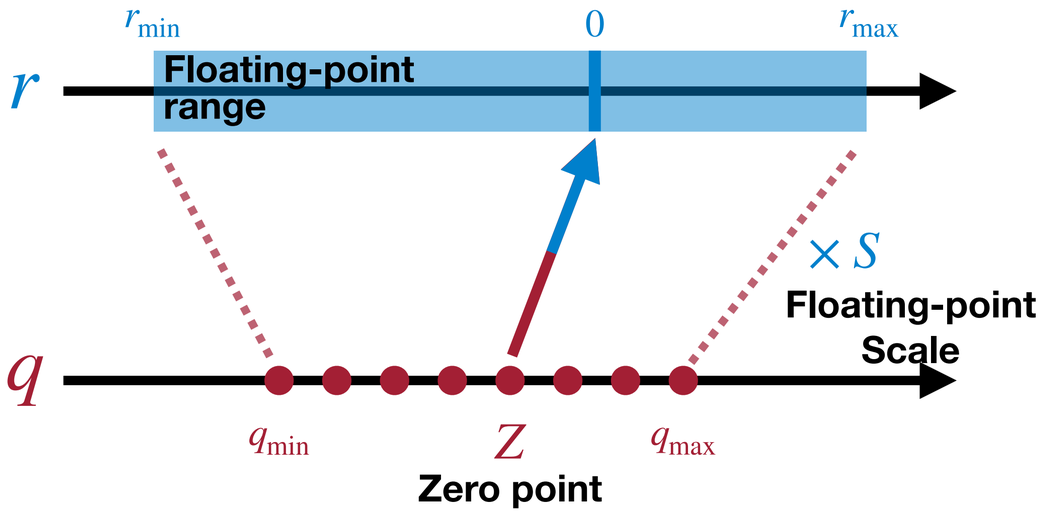
Mixed-Precision Quantized MobileNetV2

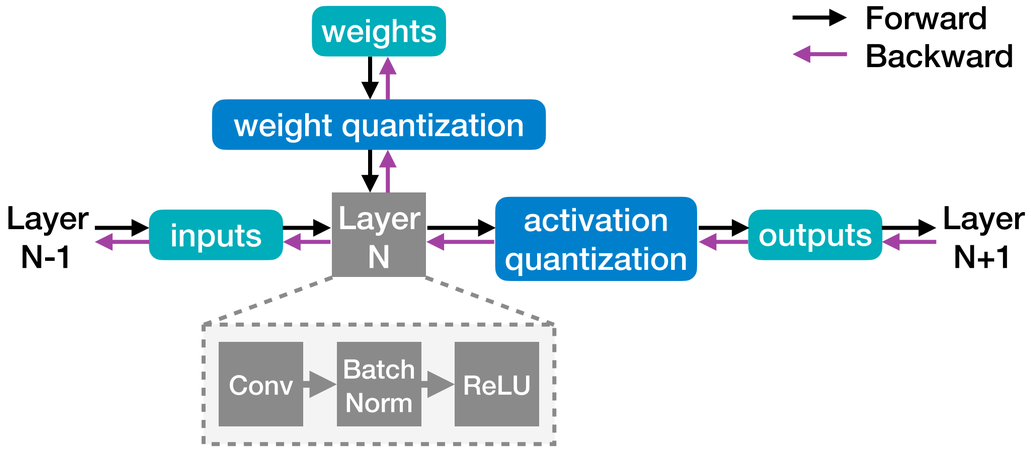
HAQ: Hardware-Aware Automated Quantization with Mixed Precision [Wang *et al.*, CVPR 2019]

Summary of Today's Lecture

In this lecture, we

1. Reviewed Linear Quantization.
2. Introduced **Post-Training Quantization (PTQ)** that quantizes an already-trained floating-point neural network model.
   - Per-tensor *vs.* per-channel *vs.* group quantization
   - How to determine dynamic range for quantization
3. Introduced **Quantization-Aware Training (QAT)** that emulates inference-time quantization during the training/fine-tuning.
   - Straight-Through Estimator (STE)
4. Introduced **binary and ternary** quantization.
5. Introduced automatic **mixed-precision** quantization.





References

1. Deep Compression [Han et al., ICLR 2016]
2. Neural Network Distiller: https://intellabs.github.io/distiller/algo_quantization.html
3. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference [Jacob et al., CVPR 2018]
4. Data-Free Quantization Through Weight Equalization and Bias Correction [Markus et al., ICCV 2019]
5. Post-Training 4-Bit Quantization of Convolution Networks for Rapid-Deployment [Banner et al., NeurIPS 2019]
6. 8-bit Inference with TensorRT [Szymon Migacz, 2017]
7. Quantizing Deep Convolutional Networks for Efficient Inference: A Whitepaper [Raghuraman Krishnamoorthi, arXiv 2018]
8. Neural Networks for Machine Learning [Hinton et al., Coursera Video Lecture, 2012]
9. Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation [Bengio, arXiv 2013]
10. Binarized Neural Networks: Training Deep Neural Networks with Weights and Activations Constrained to +1 or –1. [Courbariaux et al., Arxiv 2016]
11. DoReFa-Net: Training Low Bitwidth Convolutional Neural Networks with Low Bitwidth Gradients [Zhou et al., arXiv 2016]
12. PACT: Parameterized Clipping Activation for Quantized Neural Networks [Choi et al., arXiv 2018]
13. WRPN: Wide Reduced-Precision Networks [Mishra et al., ICLR 2018]
14. Towards Accurate Binary Convolutional Neural Network [Lin et al., NeurIPS 2017]
15. Incremental Network Quantization: Towards Lossless CNNs with Low-precision Weights [Zhou et al., ICLR 2017]
16. HAQ: Hardware-Aware Automated Quantization with Mixed Precision [Wang et al., CVPR 2019]